WOJSKOWA AKADEMIA TECHNICZNA
im. Jarosława Dąbrowskiego

WYDZIAŁ CYBERNETYKI



PRACA DYPLOMOWA INŻYNIERSKA

KRYPTOLOGIA I CYBERBEZPIECZEŃSTWO

Ocena wpływu agentów sztucznej inteligencji na bezpieczeństwo informacyjne organizacji

Karol Narożniak

Promotor
dr inż. Rafał Kasprzyk

Warszawa 2026

Spis treści

Wstęp ... 5
Rozdział 1. Założenia i metodyka pracy ... 7
1.1. Przedmiot badań ... 7
1.2. Cel pracy ... 7
1.3. Problem i pytania badawcze ... 8
1.4. Teza i hipotezy badawcze ... 8
1.5. Zakres pracy ... 9
1.6. Metoda realizacji pracy ... 10
Rozdział 2. Podstawy teoretyczne i charakterystyka zastosowanych technologii ... 11
2.1. Duże modele językowe ... 11
2.2. Systemy agentowe ... 12
2.3. Agentowe zintegrowane środowiska programistyczne ... 13
2.4. Używanie narzędzi przez model językowy ... 13
2.5. Model Context Protocol ... 14
2.6. Zagrożenia wynikające z manipulacji kontekstem ... 15
2.7. Delegowanie uprawnień i zasada najmniejszych uprawnień ... 15
2.8. Deterministyczna kontrola operacji ... 16
2.9. Izolacja środowiska wykonawczego ... 17
2.10. Segmentacja sieci i OPNsense ... 17
2.11. LiteLLM jako brama dostępu do modelu ... 18
2.12. Monitoring zdarzeń systemowych ... 18
2.13. Gromadzenie logów za pomocą Fluent Bit ... 19
2.14. Monitoring sieciowy: PCAP i Zeek ... 19
2.15. Synchronizacja czasu ... 20
2.16. Korelacja telemetrii i atrybucja działań ... 20
2.17. Dane-canary jako sposób potwierdzania skutku ... 21
2.18. Bezpieczeństwo laboratorium badawczego ... 21
2.19. Podsumowanie ... 22
Rozdział 3. Projekt i implementacja środowiska badawczego ... 23
3.1. Cel i zakres implementacji ... 23
3.2. Wymagania dla laboratorium ... 23
3.2.1. Wymagania funkcjonalne ... 23
3.2.2. Wymagania pozafunkcjonalne ... 24
3.3. Architektura logiczna laboratorium ... 24
3.4. Architektura maszyn wirtualnych ... 25
3.5. Projekt sieci i segmentacja ... 25
3.6. Implementacja maszyny agenta ... 26
3.7. Implementacja dostępu do modelu przez LiteLLM ... 27
3.8. Implementacja serwerów MCP ... 28
3.8.1. Lokalny serwer MCP ... 28
3.8.2. Zdalny serwer MCP ... 29
3.9. Implementacja mechanizmu kontroli operacji ... 29
3.9.1. Normalizacja parametrów ... 30

3.9.2. Rejestrowanie decyzji 31
3.10. Implementacja kontrolowanych celów testowych 31
3.11. Monitoring hosta agenta 32
3.12. Centralne gromadzenie logów 33
3.13. Monitoring ruchu sieciowego 33
3.14. Synchronizacja czasu 34
3.15. Identyfikacja przebiegów i korelacja zdarzeń 34
3.16. Procedura uruchomienia pojedynczej próby 35
3.17. Testy implementacji laboratorium 36
3.17.1. Testy mechanizmu kontrolnego 36
3.17.2. Testy monitoringu 37
3.17.3. Testy segmentacji 37
3.17.4. Testy odtwarzalności 37
3.18. Kryteria odbioru implementacji 37
3.19. Sposób prezentowania kodu i konfiguracji 38
3.20. Ograniczenia implementacji 39
Rozdział 4. Przykłady działania aplikacji i realizacja scenariuszy 40
4.1. Cel rozdziału 40
4.2. Sposób dokumentowania działania aplikacji 40
4.3. Uruchomienie środowiska i aplikacji 41
4.4. Scenariusz SC-01 – dozwolony odczyt pliku 41
4.4.1. Cel scenariusza 41
4.4.2. Polecenie i przebieg 42
4.4.3. Weryfikacja wyniku 42
4.5. Scenariusz SC-02 – operacja wymagająca potwierdzenia 43
4.5.1. Cel scenariusza 43
4.5.2. Wariant A – zatwierdzenie operacji 43
4.5.3. Wariant B – odrzucenie operacji 43
4.6. Scenariusz SC-03 – zablokowany dostęp do chronionego zasobu 44
4.6.1. Cel scenariusza 44
4.6.2. Przebieg 44
4.6.3. Potwierdzenie braku wykonania 45
4.7. Scenariusz SC-04 – użycie lokalnego serwera MCP 45
4.7.1. Cel scenariusza 45
4.7.2. Widoczność operacji 46
4.8. Scenariusz SC-05 – użycie zdalnego serwera MCP 46
4.8.1. Cel scenariusza 46
4.8.2. Korelacja telemetrii 46
4.9. Scenariusz SC-06 – próba wysłania danych-canary 47
4.9.1. Cel scenariusza 47
4.9.2. Wariant zablokowany 47
4.9.3. Wariant kontrolnie dozwolony 47
4.10. Przykład korelacji zdarzeń 48
Rozdział 5. Ocena realizacji celów i wnioski 49

5.1. Cel rozdziału ... 49
5.2. Podsumowanie wykonanych prac ... 49
5.3. Ocena realizacji celu głównego ... 50
5.4. Ocena realizacji celów szczegółowych ... 51
5.5. Odpowiedzi na pytania badawcze ... 52
5.5.1. Czy można kontrolować operacje wykonywane przez agentowe środowisko programistyczne? ... 52
5.5.2. Czy można odróżnić zamiar agenta od rzeczywiście wykonanego działania? ... 52
5.5.3. Czy pojedyncze źródło telemetrii wystarcza do oceny działania agenta? ... 53
5.5.4. Czy dane-canary pozwalają potwierdzać niepożądany przepływ informacji? ... 53
5.5.5. Jak różni się obserwowalność lokalnych i zdalnych narzędzi MCP? ... 53
5.6. Weryfikacja hipotez badawczych ... 54
5.7. Najważniejsze wnioski ... 55
5.8. Ograniczenia pracy ... 56
5.9. Kierunki dalszego rozwoju ... 57
Podsumowanie ... 58
Bibliografia ... 59
Spis tabel ... 60
Spis kodów źródłowych ... 61

Wstęp

Rozwój dużych modeli językowych doprowadził do powstania systemów, które nie ograniczają się do generowania treści, lecz mogą samodzielnie planować działania oraz wykonywać operacje za pomocą udostępnionych narzędzi. Przykładem takiego rozwiązania są agentowe zintegrowane środowiska programistyczne, w których model sztucznej inteligencji może analizować repozytorium, modyfikować pliki, uruchamiać polecenia systemowe, korzystać z usług sieciowych oraz komunikować się z zewnętrznymi serwerami narzędzi. Zwiększa to możliwości automatyzacji pracy programisty, ale jednocześnie powoduje powstanie nowych zagrożeń dla poufności, integralności i dostępności informacji.

W klasycznym środowisku programistycznym większość operacji jest bezpośrednio inicjowana przez użytkownika. W systemie agentowym pomiędzy poleceniem użytkownika a rzeczywistym działaniem występuje dodatkowa warstwa decyzyjna w postaci modelu sztucznej inteligencji. Model interpretuje zadanie, analizuje dostępny kontekst, wybiera narzędzia oraz formułuje ich parametry. Wynikająca z tego operacja może zostać wykonana bez pełnego zrozumienia jej konsekwencji przez użytkownika. Szczególne znaczenie ma to w sytuacji, w której agent działa z wykorzystaniem tożsamości, uprawnień i danych uwierzytelniających osoby korzystającej ze środowiska.

Źródłem niepożądanego działania agenta nie musi być wyłącznie błędnie sformułowane polecenie użytkownika. Na decyzję modelu mogą wpływać również dane znajdujące się w repozytorium, dokumentacja, komentarze w kodzie źródłowym, odpowiedzi usług sieciowych, wyniki działania narzędzi oraz opisy i odpowiedzi serwerów działających zgodnie z protokołem Model Context Protocol (MCP). Elementy te mogą zawierać błędne, nieaktualne lub celowo spreparowane instrukcje. Oznacza to, że zawartość przetwarzana przez agenta powinna być traktowana jako potencjalnie niezaufana, nawet jeśli pochodzi z zasobu, który w tradycyjnym procesie programistycznym nie był uznawany za źródło instrukcji wykonawczych.

Dodatkową trudność stanowi rozróżnienie pomiędzy odpowiedzią wygenerowaną przez model, próbą wywołania narzędzia, wykonaniem operacji a jej rzeczywistym skutkiem. Samo wygenerowanie przez model żądania odczytu pliku nie oznacza jeszcze, że plik został odczytany. Analogicznie, próba przesłania danych do usługi sieciowej nie musi zakończyć się skuteczną transmisją. Wiarygodna ocena działania agenta wymaga zatem obserwacji zarówno warstwy aplikacyjnej, jak i zdarzeń zachodzących w systemie operacyjnym oraz komunikacji sieciowej.

W pracy przyjęto założenie, że agent sztucznej inteligencji powinien być traktowany jako odrębny podmiot bezpieczeństwa, nawet jeżeli technicznie wykonuje działania w imieniu użytkownika. Takie podejście wymaga ograniczenia zakresu delegowanych uprawnień, kontroli wykonywanych operacji, izolacji środowiska wykonawczego oraz zapewnienia śladu audytowego umożliwiającego przypisanie zaobserwowanych skutków do określonego zadania, przebiegu agenta i wywołania narzędzia. Decyzja modelu probabilistycznego nie powinna być w tym ujęciu utożsamiana z decyzją bezpieczeństwa. Dopuszczalność operacji powinna być oceniana przez niezależny, deterministyczny mechanizm kontrolny.

Celem pracy jest zaprojektowanie, wykonanie i eksperymentalna ocena mechanizmu ochrony agentowego środowiska programistycznego, opartego na rozdzieleniu kontekstu

działania użytkownika i agenta, kontroli operacji, izolacji środowiska wykonawczego oraz korelacji telemetrii aplikacyjnej, hosta i sieciowej.

Rozdział pierwszy przedstawia przedmiot i zakres badań, cele, pytania badawcze, hipotezy oraz metodę realizacji pracy. Rozdział drugi zawiera podstawy teoretyczne i charakterystykę zastosowanych technologii.

W rozdziale trzecim opisano projekt i implementację środowiska badawczego, a w rozdziale czwartym przedstawiono scenariusze oraz sposób dokumentowania działania środowiska. Rozdział piąty obejmuje ocenę realizacji celów, weryfikację hipotez, wnioski, ograniczenia i kierunki dalszego rozwoju. Pracę zamyka podsumowanie.

Rozdział 1. Założenia i metodyka pracy

1.1. Przedmiot badań

Przedmiotem pracy jest bezpieczeństwo agentowego środowiska programistycznego Kilo Code, działającego jako rozszerzenie środowiska Visual Studio Code. Badania przeprowadzono w odizolowanym laboratorium zbudowanym z maszyn wirtualnych uruchomionych w VMware Workstation Pro 26. Agent komunikował się z chmurowym modelem GPT-4.6 przez kontrolowany punkt pośredniczący, a operacje wykonawcze realizował za pomocą narzędzi lokalnych oraz serwerów MCP. Środowisko uzupełniono o mechanizmy kontroli operacji i gromadzenia telemetrii aplikacyjnej, hosta oraz sieciowej.

W ramach części praktycznej utworzono syntetyczne repozytorium, kontrolowane usługi sieciowe, lokalny i zdalny serwer MCP oraz dane testowe pozwalające jednoznacznie wykryć odczyt albo przesłanie chronionej informacji. Zaimplementowano również mechanizm oceniający żądania użycia narzędzi zgodnie z regułami allow, ask i deny. Mechanizm uwzględniał między innymi rodzaj operacji, parametry wywołania, ścieżkę zasobu oraz docelową usługę sieciową.

Badanie przeprowadzono dla dwóch konfiguracji; bazowej oraz zabezpieczonej. W konfiguracji bazowej agent dysponował szerszym kontekstem wykonania oraz mniejszą liczbą ograniczeń. W konfiguracji zabezpieczonej zastosowano wydzielone środowisko, ograniczenia uprawnień, kontrolę dostępu do narzędzi i usług sieciowych oraz korelację zdarzeń pochodzących z różnych warstw monitoringu. Porównanie konfiguracji pozwoliło ocenić nie tylko możliwość zablokowania określonej operacji, lecz także wpływ zastosowanych zabezpieczeń na zdolność agenta do wykonywania prawidłowych zadań.

1.2. Cel pracy

Głównym celem pracy było zaprojektowanie, wykonanie i eksperymentalna ocena mechanizmu ochrony agentowego środowiska programistycznego. Mechanizm ten oparto na rozdzieleniu kontekstu działania użytkownika i agenta, deterministycznej kontroli operacji, izolacji środowiska wykonawczego oraz korelacji telemetrii aplikacyjnej, hosta i sieciowej.

Realizacja celu głównego wymagała osiągnięcia następujących celów szczegółowych:

- rozpoznania sposobu komunikacji agentowego środowiska Kilo Code z modelem językowym, narzędziami lokalnymi i serwerami MCP;
- opracowania modelu zagrożeń uwzględniającego niezaufane polecenia, zawartość repozytorium, wyniki narzędzi, odpowiedzi usług sieciowych oraz dane zwracane przez serwery MCP;
- zaprojektowania odtwarzalnego laboratorium składającego się z wydzielonych maszyn wirtualnych, stref sieciowych, kontrolowanych usług oraz centralnej warstwy monitoringu;
- przygotowania syntetycznego repozytorium, danych-canary i kontrolowanych usług pozwalających potwierdzić rzeczywiste skutki operacji agenta;
- implementacji lokalnego oraz zdalnego serwera MCP udostępniających narzędzia wykorzystywane w scenariuszach badawczych;
- implementacji deterministycznego mechanizmu kontrolującego operacje agenta na podstawie reguł allow, ask i deny;

- wdrożenia monitoringu umożliwiającego rejestrację zdarzeń aplikacyjnych, procesów systemowych, operacji na plikach oraz komunikacji sieciowej;
- opracowania wspólnego formatu zdarzeń i mechanizmu korelacji pozwalającego odtworzyć sekwencję działań od polecenia użytkownika do skutku w środowisku;
- porównania potencjalnego wpływu operacji wykonywanych w szerokim kontekście użytkownika i w wydzielonym, ograniczonym kontekście agenta;
- sprawdzenia skuteczności mechanizmów kontrolnych wobec prawidłowych zadań oraz scenariuszy zawierających bezpośrednią lub pośrednią próbę wpłynięcia na działanie agenta;
- oceny wpływu zastosowanych zabezpieczeń na użyteczność środowiska, w szczególności na poprawność wykonania zadań, liczbę błędnych blokad i czas realizacji;
- sformułowania zaleceń dotyczących bezpiecznego wdrażania agentowych środowisk programistycznych w organizacji.

Z komentarzem [R1]: DO UZUPEŁNIENIA PO BADANIACH: po zakończeniu eksperymentów należy sprawdzić każdy cel szczegółowy i usunąć albo przeformułować cele, których faktycznie nie zrealizowano. Celów nie należy deklarować jako osiągniętych wyłącznie na podstawie przygotowania projektu architektury.

1.3. Problem i pytania badawcze

Główny problem badawczy sprowadza się do ustalenia, w jakim stopniu rozdzielenie kontekstu wykonania agenta, deterministyczna kontrola jego operacji oraz korelacja telemetrii ograniczają ryzyko niepożądanego działania agentowego środowiska programistycznego i zwiększają możliwość przypisania rzeczywistych skutków do działań agenta.

Na tej podstawie sformułowano następujące pytania badawcze:

- Jakie działania agentowego środowiska programistycznego są widoczne w telemetrii aplikacyjnej, hosta i sieciowej?
- Czy pojedyncza warstwa monitoringu pozwala jednoznacznie odtworzyć przebieg działania agenta i potwierdzić jego rzeczywisty skutek?
- Jak zakres uprawnień i kontekst wykonania wpływają na potencjalne konsekwencje niepożądanego działania agenta?
- Czy deterministyczna kontrola nazw narzędzi i parametrów ich wywołań ogranicza możliwość wykonania niedozwolonych operacji?
- Czy korelacja zdarzeń z kilku warstw umożliwia dokładniejsze przypisanie skutków do działania agenta niż analiza każdego źródła osobno?
- Jaki wpływ mają zastosowane zabezpieczenia na prawidłowe wykonywanie zadań przez agentowe środowisko programistyczne?

1.4. Teza i hipotezy badawcze

W pracy przyjęto tezę, zgodnie z którą bezpieczne wdrożenie agentów sztucznej inteligencji wymaga traktowania ich jako odrębnych podmiotów bezpieczeństwa, posiadających ograniczony zakres delegowanych uprawnień i możliwy do wyodrębnienia ślad audytowy. Samo wykorzystanie uprawnień użytkownika oraz rejestrowanie zdarzeń wyłącznie na poziomie aplikacji nie zapewnia dostatecznej kontroli ani możliwości wiarygodnego przypisania skutków działania.

Weryfikacji poddano następujące hipotezy:

- **H1:** telemetria pochodząca wyłącznie z agentowego środowiska programistycznego nie pozwala na pełne potwierdzenie rzeczywistych skutków operacji agenta;

- **H2:** wykonanie tego samego niepożądanego działania w szerokim kontekście użytkownika może powodować większy skutek niż wykonanie go w wydzielonym kontekście o ograniczonych uprawnieniach;
- **H3:** deterministyczna kontrola narzędzi i parametrów ich wywołań ogranicza liczbę skutecznie wykonanych operacji niedozwolonych, również w przypadku manipulacji kontekstem przetwarzanym przez model;
- **H4: korelacja telemetrii aplikacyjnej, hosta i sieciowej umożliwia pełniejszą rekonstrukcję działania agenta niż wykorzystanie dowolnego z tych źródeł osobno.**

Z komentarzem [R2]: DO UZUPEŁNIENIA PO BADANIACH: hipotez nie należy zmieniać tylko dlatego, że wyniki ich nie potwierdziły. Po analizie trzeba natomiast dopisać w podsumowaniu, czy każda hipoteza została potwierdzona, częściowo potwierdzona, czy odrzucona, oraz wskazać odpowiednie wyniki i tabele.

1.5. Zakres pracy

Zakres pracy obejmuje jedno agentowe środowisko programistyczne, jeden podstawowy model chmurowy oraz przygotowane przez autora środowisko laboratoryjne. Badaniu podlega komunikacja Kilo Code z modelem, lokalnymi narzędziami, lokalnym serwerem MCP wykorzystującym standardowe wejście i wyjście oraz zdalnym serwerem MCP komunikującym się przez sieć.

Analiza obejmuje próby odczytu i modyfikacji plików, uruchamiania procesów, wykorzystania narzędzi oraz komunikacji z kontrolowanymi usługami sieciowymi. Oceniane są zarówno działania prawidłowe, jak i operacje niedozwolone inicjowane bezpośrednio albo wynikające z przetwarzania spreparowanego kontekstu. Wszystkie testy prowadzone są na danych syntetycznych i usługach utworzonych na potrzeby laboratorium.

Praca nie obejmuje wdrożenia produkcyjnego, oceny wszystkich dostępnych agentowych środowisk programistycznych, testowania wszystkich modeli językowych ani kompleksowej analizy zgodności prawnej. Wyników nie należy zatem bezpośrednio uogólniać na wszystkie systemy agentowe. Mogą one jednak stanowić podstawę do identyfikacji mechanizmów bezpieczeństwa, które powinny być rozważane podczas wdrażania podobnych rozwiązań w organizacji.

Z komentarzem [R3]: DO UZUPEŁNIENIA PO LABORATORIUM: wymienić ostatecznie przetestowane rodzaje operacji, narzędzia Kilo Code i MCP oraz źródła telemetrii. Należy usunąć elementy, które były zaplanowane, ale nie zostały uruchomione albo zweryfikowane.

1.6. Metoda realizacji pracy

Pracę zrealizowano metodą projektowo-eksperymentalną. W pierwszym etapie przeanalizowano architekturę agentowego środowiska programistycznego i opracowano model zagrożeń. Następnie zaprojektowano oraz zbudowano laboratorium składające się z odseparowanych maszyn wirtualnych, kontrolowanych usług i warstwy monitoringu.

W kolejnym etapie zaimplementowano komponenty niezbędne do przeprowadzenia badań: serwery MCP, mechanizm kontroli operacji, zasoby testowe oraz moduł normalizacji i korelacji zdarzeń. Następnie przeprowadzono scenariusze obejmujące prawidłowe zadania oraz próby wykonania działań niedozwolonych. Każdy scenariusz realizowano w ustalonych konfiguracjach bezpieczeństwa, a uzyskane zdarzenia analizowano z perspektywy aplikacyjnej, hosta i sieciowej.

Skuteczność zabezpieczeń oceniono na podstawie odsetka zablokowanych operacji niedozwolonych, liczby prawidłowo wykonanych zadań, liczby błędnych blokad, kompletności rejestracji i atrybucji zdarzeń oraz czasu realizacji zadań. Oddzielnie rejestrowano decyzję modelu, próbę wywołania narzędzia, decyzję mechanizmu kontrolnego, wykonanie operacji oraz jej potwierdzony skutek.

Z komentarzem [R4]: DO UZUPEŁNIENIA PO BADANIACH: wpisać ostateczne definicje metryk, liczbę powtórzeń, metodę agregacji wyników oraz sposób postępowania z próbami niekompletnymi. Jeżeli nie zmierzono czasu lub błędnych blokad, wskaźniki te należy usunąć z opisu metody.

Rozdział 2. Podstawy teoretyczne i charakterystyka zastosowanych technologii

Niniejszy rozdział przedstawia podstawy teoretyczne działania agentowych środowisk programistycznych oraz technologie wykorzystane do budowy środowiska badawczego. Omówiono sposób współpracy modelu językowego z narzędziami, protokół Model Context Protocol, zagadnienia delegowania uprawnień, izolacji i kontroli operacji, a także mechanizmy monitorowania zdarzeń aplikacyjnych, systemowych i sieciowych.

Rozdział nie opisuje szczegółowej konfiguracji laboratorium ani implementacji opracowanych komponentów. Informacje takie jak adresacja IP, reguły zapory sieciowej, konfiguracja maszyn wirtualnych, format autorskich polityk i struktura modułu korelacyjnego powinny zostać przedstawione w rozdziale projektowo-implementacyjnym.

2.1. Duże modele językowe

Duże modele językowe, ang. *Large Language Models* (LLM), są modelami uczenia maszynowego przetwarzającymi i generującymi dane tekstowe. Typowy model językowy otrzymuje sekwencję tokenów i wyznacza prawdopodobieństwo wystąpienia kolejnych tokenów. Token nie musi odpowiadać całemu słowu. Może reprezentować słowo, jego fragment, znak interpunkcyjny albo inny element przetwarzanego tekstu.

Działanie współczesnych dużych modeli językowych opiera się przede wszystkim na architekturze transformera. Jej istotnym elementem jest mechanizm uwagi, pozwalający określić zależności pomiędzy elementami sekwencji niezależnie od ich odległości w tekście. Model może dzięki temu uwzględniać między innymi wcześniejsze polecenia, fragmenty kodu źródłowego, dokumentację oraz wyniki działania narzędzi.

Dane przekazane modelowi podczas pojedynczego wywołania tworzą jego kontekst. W agentowym środowisku programistycznym kontekst może obejmować:

- polecenie użytkownika;
- instrukcje systemowe i konfiguracyjne;
- historię interakcji;
- zawartość plików;
- fragmenty kodu źródłowego;
- opis dostępnych narzędzi i ich parametrów;
- odpowiedzi serwerów MCP;
- wyniki wykonanych poleceń;
- komunikaty błędów i inne dane zwrócone przez środowisko.

Model nie analizuje pochodzenia każdego fragmentu tekstu w taki sam sposób jak deterministyczny mechanizm bezpieczeństwa. Treść pochodząca z niezaufanego pliku może zostać przetworzona razem z właściwym poleceniem użytkownika. Powoduje to ryzyko potraktowania danych jako instrukcji mających wpływ na dalsze działanie.

Odpowiedzi modelu mają charakter probabilistyczny. Nawet dla zbliżonych danych wejściowych model może generować odmienne odpowiedzi albo wybierać różne sekwencje działań. Na wynik wpływają między innymi wersja modelu, zawartość kontekstu, parametry generowania oraz aktualny stan interakcji. Z tego względu decyzja modelu o użyciu narzędzia nie powinna być traktowana jako deterministyczna decyzja bezpieczeństwa.

W pracy wykorzystano chmurowy model z rodziny GPT udostępniany za pomocą interfejsu API. Model odpowiadał za interpretację zadania, analizę kontekstu i proponowanie kolejnych działań. Nie pełnił natomiast funkcji mechanizmu autoryzacyjnego. Ostateczna możliwość wykonania operacji zależała od uprawnień systemowych, konfiguracji sieci i dodatkowej warstwy kontroli.

Z komentarzem [R5]: DO UZUPEŁNIENIA PRZED FINALIZACJĄ: podać dokładne oznaczenie modelu zwracane przez API, datę dostępu, wykorzystywany punkt końcowy API oraz parametry wpływające na generowanie. Nazwę „GPT-4.6" należy pozostawić tylko wtedy, gdy odpowiada rzeczywistemu identyfikatorowi lub oficjalnej nazwie modelu.

2.2. Systemy agentowe

Klasyczny model językowy generuje odpowiedź na podstawie przekazanego kontekstu. System agentowy rozszerza ten mechanizm o możliwość planowania kolejnych kroków, wybierania narzędzi, analizowania wyników ich działania i kontynuowania zadania. Agent nie jest zatem wyłącznie modelem. Jest systemem składającym się z modelu, mechanizmu zarządzania kontekstem, zestawu narzędzi, warstwy wykonawczej i logiki sterującej.

Uproszczony cykl działania agenta można przedstawić następująco:

- odebranie celu określonego przez użytkownika;
- zebranie informacji o bieżącym kontekście;
- wybranie kolejnej czynności;
- przygotowanie nazwy narzędzia i parametrów jego wywołania;
- ocena dopuszczalności operacji;
- wykonanie dozwolonego działania;
- przekazanie wyniku do modelu;
- ponowienie cyklu aż do zakończenia zadania albo wystąpienia warunku zatrzymania.

Taki cykl przypomina podejście łączące rozumowanie z działaniem, w którym kolejne decyzje są podejmowane na podstawie wyników wcześniejszych operacji. W praktyce szczegóły procesu decyzyjnego mogą nie być dostępne dla użytkownika. Obserwowalne są przede wszystkim polecenia przekazane modelowi, wybrane narzędzia, parametry wywołań, odpowiedzi modelu i skutki w środowisku.

Agentowe środowisko programistyczne może otrzymać dostęp do szczególnie istotnych funkcji, takich jak:

- odczytywanie i modyfikowanie plików;
- przeszukiwanie repozytorium;
- uruchamianie poleceń systemowych;
- wykonywanie skryptów i testów;
- instalowanie zależności;
- komunikowanie się z usługami sieciowymi;
- korzystanie z repozytoriów kodu;
- wywoływanie narzędzi udostępnionych przez serwery MCP.

Z punktu widzenia bezpieczeństwa kluczowe jest rozróżnienie pomiędzy decyzją modelu a operacją wykonaną przez system. Model może jedynie wygenerować żądanie użycia narzędzia. Dopiero warstwa wykonawcza przekształca to żądanie w operację na pliku, uruchomienie procesu albo transmisję sieciową.

2.3. Agentowe zintegrowane środowiska programistyczne

Zintegrowane środowisko programistyczne, ang. *Integrated Development Environment* (IDE), łączy narzędzia potrzebne do tworzenia, analizowania, uruchamiania i testowania oprogramowania. Rozszerzenie go o agenta AI umożliwia wykonywanie złożonych zadań za pomocą poleceń w języku naturalnym.

Agentowe IDE może analizować strukturę projektu i samodzielnie dobierać pliki związane z określonym zadaniem. W zależności od konfiguracji może również proponować lub bezpośrednio wprowadzać zmiany w kodzie, uruchamiać polecenia oraz korzystać z usług zewnętrznych. Powstaje w ten sposób warstwa automatyzacji pośrednicząca pomiędzy intencją użytkownika a systemem operacyjnym.

Takie rozwiązanie może zwiększać efektywność programowania, lecz jednocześnie rozszerza powierzchnię ataku. Elementami tej powierzchni są w szczególności:

- kontekst przekazywany modelowi;
- konfiguracja agenta;
- repozytorium i znajdujące się w nim instrukcje;
- mechanizm zatwierdzania operacji;
- narzędzia lokalne;
- serwery MCP;
- dane uwierzytelniające;
- interfejs komunikacji z modelem;
- system operacyjny, na którym działa agent;
- połączenia sieciowe inicjowane przez narzędzia.

W badaniach wykorzystano Kilo Code działające w środowisku Visual Studio Code. Kilo Code udostępnia tryby przeznaczone między innymi do tworzenia kodu, projektowania rozwiązania, diagnozowania błędów i odpowiadania na pytania. Może również współpracować z serwerami MCP oraz różnymi dostawcami modeli językowych. **Błąd! Nieprawidłowy odsyłacz typu hiperłącze.**

Kilo Code stanowiło badany interfejs agentowy, natomiast Visual Studio Code zapewniało środowisko edycji projektu. Takie rozróżnienie jest istotne: nie każda operacja wykonywana przez Visual Studio Code jest działaniem agenta, a nie każde żądanie wygenerowane przez agenta musi zostać wykonane przez system.

Z komentarzem [R6]: DO UZUPEŁNIENIA PO ZBUDOWANIU LABORATORIUM: podać dokładne wersje Visual Studio Code i Kilo Code, wykorzystany tryb działania, ustawienia automatycznego zatwierdzania oraz ostateczną listę funkcji udostępnionych agentowi.

2.4. Używanie narzędzi przez model językowy

Interfejs API modelu może umożliwiać przekazanie opisów funkcji lub narzędzi, z których model może korzystać. Opis narzędzia obejmuje zazwyczaj jego nazwę, przeznaczenie oraz schemat argumentów. Na tej podstawie model może wygenerować ustrukturyzowane żądanie wywołania.

Przykładowe żądanie może zawierać nazwę narzędzia read_file i parametr określający ścieżkę pliku. Samo wygenerowanie takiej struktury nie powoduje odczytania danych. Aplikacja obsługująca model musi odebrać żądanie, zweryfikować je, wywołać odpowiednią funkcję i przekazać wynik z powrotem do modelu.

W procesie należy zatem rozróżnić następujące zdarzenia:

Tab. 2.1. Etapy działania agenta i ich obserwowalność

Etap	Znaczenie
Intencja modelu	Model proponuje użycie określonego narzędzia
Żądanie operacji	Aplikacja otrzymuje nazwę narzędzia i jego parametry
Decyzja kontrolna	Operacja zostaje dopuszczona, zatrzymana do zatwierdzenia albo odrzucona
Próba wykonania	Warstwa wykonawcza podejmuje próbę realizacji operacji
Skutek	Następuje rzeczywisty odczyt, zapis, uruchomienie procesu lub transmisja
Wynik dla modelu	Rezultat operacji zostaje włączony do dalszego kontekstu

Źródło: [do wskazania przez autora].

Rozróżnienie to ma podstawowe znaczenie dla interpretacji telemetrii. Log aplikacyjny informujący o wybraniu narzędzia nie potwierdza jeszcze rzeczywistego skutku. Skutek powinien zostać potwierdzony w systemie operacyjnym, usłudze docelowej albo ruchu sieciowym.

2.5. Model Context Protocol

Model Context Protocol (MCP) jest protokołem umożliwiającym ujednoliconą komunikację aplikacji wykorzystujących modele językowe z zewnętrznymi źródłami danych i narzędziami. W architekturze MCP wyróżnia się hosta, klienta oraz serwer. Host zarządza interakcją użytkownika i modelu, klient utrzymuje połączenie z określonym serwerem, a serwer udostępnia wybrane możliwości.

Serwer MCP może udostępniać między innymi:

- narzędzia, czyli operacje możliwe do wywołania;
- zasoby, czyli dane możliwe do pobrania;
- szablony promptów;
- informacje o swoich możliwościach i obsługiwanych funkcjach.

MCP wykorzystuje ustrukturyzowane komunikaty oparte na JSON-RPC. Pozwala to odseparować opis operacji od konkretnej implementacji narzędzia. Sam protokół nie gwarantuje jednak, że udostępnione narzędzie jest bezpieczne albo że jego wynik jest wiarygodny. Kontrola zaufania i uprawnień pozostaje zadaniem systemu wykorzystującego MCP.

W laboratorium uwzględniono dwa rodzaje transportu:

- stdio, stosowany do komunikacji z lokalnym serwerem MCP;
- Streamable HTTP, wykorzystywany do komunikacji z serwerem zdalnym.

W transporcie stdio klient uruchamia serwer jako proces potomny i komunikuje się z nim przez standardowe wejście i wyjście. Połączenie nie wykorzystuje protokołu IP ani portu sieciowego. Z tego względu jego przebieg nie może zostać bezpośrednio odtworzony wyłącznie na podstawie monitorowania sieci. Konieczne jest wykorzystanie logów aplikacyjnych i zdarzeń systemu operacyjnego.

W transporcie Streamable HTTP serwer działa jako niezależna usługa. Komunikaty klienta są przekazywane przede wszystkim za pomocą żądań HTTP POST, natomiast odpowiedzi strumieniowe mogą wykorzystywać Server-Sent Events. Transport może korzystać z mechanizmów uwierzytelniania dostępnych dla usług HTTP. Aktualna specyfikacja MCP wyróżnia stdio oraz Streamable HTTP jako standardowe mechanizmy transportowe. **Błąd! Nieprawidłowy odsyłacz typu hiperłącze.**

Z punktu widzenia bezpieczeństwa serwer MCP stanowi granicę zaufania. Jego opis narzędzia, odpowiedź oraz dane zwracane przez narzędzie mogą wpływać na kolejne decyzje agenta. Dotyczy to zarówno serwera złośliwego, jak i serwera poprawnego, który zwraca dane pochodzące z niezaufanego źródła.

Z komentarzem [R7]: DO UZUPEŁNIENIA: wskazać wersję specyfikacji MCP faktycznie obsługiwaną przez zastosowaną wersję Kilo Code i biblioteki serwerowej. Nie należy automatycznie przyjmować najnowszej wersji specyfikacji dostępnej podczas końcowej redakcji.

2.6. Zagrożenia wynikające z manipulacji kontekstem

Jednym z zagrożeń dla systemów agentowych jest *prompt injection*, czyli umieszczenie w przetwarzanych danych treści wpływających na zachowanie modelu. W bezpośrednim wariancie użytkownik sam przekazuje modelowi szkodliwą instrukcję. W wariancie pośrednim instrukcja znajduje się w zewnętrznym zasobie odczytywanym podczas realizacji prawidłowego zadania.

Źródłem pośredniej manipulacji mogą być:

- pliki tekstowe i dokumentacja;
- komentarze w kodzie;
- strony internetowe;
- opisy problemów i zgłoszeń;
- wyniki wyszukiwania;
- odpowiedzi interfejsów API;
- dane zwracane przez serwer MCP;
- nazwy plików i metadane;
- komunikaty błędów wygenerowane przez usługę.

Podstawowym problemem nie jest samo występowanie złośliwego tekstu, lecz możliwość przekształcenia jego treści w działanie posiadające skutek poza modelem. Jeżeli agent nie ma dostępu do narzędzi, manipulacja może wpłynąć głównie na treść odpowiedzi. Jeżeli natomiast może odczytywać pliki, uruchamiać procesy albo wysyłać dane, konsekwencją może być wykonanie operacji w systemie.

Nie należy zatem opierać bezpieczeństwa wyłącznie na poleceniu nakazującym modelowi ignorowanie niezaufanych instrukcji. Tego rodzaju instrukcja może ograniczyć ryzyko, lecz pozostaje częścią probabilistycznego procesu interpretacji. Ochrona operacji o istotnych skutkach wymaga również kontroli poza modelem.

2.7. Delegowanie uprawnień i zasada najmniejszych uprawnień

Agent wykonuje działania za pośrednictwem procesów i usług działających z określonymi uprawnieniami. Jeżeli uruchomiono go w kontekście użytkownika, może uzyskać dostęp do tych zasobów, które są dostępne dla konta użytkownika oraz procesu środowiska programistycznego.

Delegowanie uprawnienia oznacza przekazanie innemu podmiotowi możliwości wykonania określonego działania. W przypadku agenta delegacja powinna być ograniczona co najmniej pod względem:

- rodzaju operacji;
- zasobu, którego dotyczy działanie;
- czasu obowiązywania;
- docelowej usługi;
- zakresu danych;
- kontekstu konkretnego zadania.

Zasada najmniejszych uprawnień zakłada przydzielanie wyłącznie takich praw, które są niezbędne do wykonania określonego zadania. Agent przeznaczony do analizy kodu nie musi na przykład mieć dostępu do całego katalogu użytkownika, kluczy SSH, historii przeglądarki ani dowolnych usług internetowych.

Dodatkowym założeniem jest kompletność pośredniczenia, ang. *complete mediation*. Każda operacja dotycząca chronionego zasobu powinna podlegać kontroli, a wcześniejsze dopuszczenie podobnej operacji nie powinno automatycznie oznaczać zgody na wszystkie kolejne działania. W praktyce jest to trudne, jeśli agent posiada kilka alternatywnych dróg osiągnięcia tego samego skutku. Blokada narzędzia read_file będzie nieskuteczna, jeżeli agent nadal może uruchomić polecenie systemowe odczytujące ten sam plik.

2.8. Deterministyczna kontrola operacji

W pracy zastosowano koncepcję niezależnej warstwy kontrolującej żądania użycia narzędzi. Warstwa ta nie interpretuje celu zadania w taki sam sposób jak model, lecz porównuje nazwę operacji i jej parametry z wcześniej określonymi regułami.

Dla każdej operacji możliwe są trzy podstawowe wyniki:

- allow – operacja może zostać wykonana automatycznie;
- ask – wymagane jest świadome zatwierdzenie przez użytkownika;
- deny – operacja zostaje zablokowana.

Decyzja może zależeć między innymi od:

- nazwy narzędzia;
- ścieżki źródłowej i docelowej;
- rodzaju operacji na pliku;
- wykonywanego programu;
- argumentów polecenia;
- adresu docelowego;
- portu i protokołu;
- identyfikatora zadania;
- bieżącego trybu eksperymentu.

Model allow/ask/deny pozwala oddzielić decyzję modelu o celowości działania od decyzji o jego dopuszczalności. Mechanizm kontrolny również posiada jednak ograniczenia. Reguła analizująca wyłącznie nazwę narzędzia może nie wykryć niebezpiecznego parametru. Reguła oparta wyłącznie na ścieżce może zostać osłabiona przez dowiązania symboliczne, ścieżki

względne lub alternatywne reprezentacje tego samego zasobu. Z tego względu parametry powinny zostać znormalizowane przed oceną.

Szczegóły implementacji mechanizmu kontrolnego, struktura reguł i kolejność ich rozpatrywania zostaną przedstawione w rozdziale dotyczącym projektu i implementacji.

2.9. Izolacja środowiska wykonawczego

Izolacja ogranicza zasoby dostępne dla procesu i potencjalny wpływ niepożądanego działania. Może być realizowana na poziomie procesu, kontenera, systemu operacyjnego albo maszyny wirtualnej. Poszczególne mechanizmy różnią się zakresem separacji, wymaganiami zasobowymi i złożonością zarządzania.

W laboratorium zastosowano maszyny wirtualne. Wirtualizacja umożliwia uruchomienie kilku odseparowanych systemów gościnnych na jednym komputerze fizycznym. Każda maszyna otrzymuje wirtualne procesory, pamięć, dyski i interfejsy sieciowe. Pozwala to rozdzielić rolę agenta, usług kontrolnych, celów testowych i monitoringu.

Zaletami wykorzystania maszyn wirtualnych w badaniu są:

- wyraźne oddzielenie systemów operacyjnych;
- możliwość tworzenia migawek;
- odtwarzanie stanu początkowego;
- tworzenie niezależnych segmentów sieci;
- możliwość porównania konfiguracji;
- ograniczenie wpływu testów na system hosta.

Maszyny wirtualne nie zapewniają automatycznie pełnej izolacji. Ryzyko mogą zwiększać współdzielone foldery, schowek, przeciąganie plików, przekazywanie urządzeń oraz nieprawidłowo skonfigurowany routing. Funkcje integrujące hosta i system gościnny powinny zatem zostać ograniczone zgodnie z potrzebami eksperymentu.

Do uruchomienia laboratorium wykorzystano VMware Workstation Pro. Oprogramowanie umożliwia tworzenie maszyn wirtualnych i łączenie ich z sieciami w trybie mostkowanym, NAT, host-only albo z niestandardowymi sieciami wirtualnymi. W projekcie wykorzystano odrębne sieci VMnet, natomiast routing pomiędzy strefami powierzono wydzielonej maszynie pełniącej funkcję zapory.

Z komentarzem [R8]: DO UZUPEŁNIENIA: wpisać dokładną wersję i numer kompilacji VMware Workstation Pro odczytany z zainstalowanego programu. Należy również opisać stan funkcji integracyjnych: folderów współdzielonych, schowka, przeciągania plików i urządzeń USB.

2.10. Segmentacja sieci i OPNsense

Segmentacja sieci polega na podziale infrastruktury na strefy o odmiennym poziomie zaufania i przeznaczeniu. Komunikacja pomiędzy strefami powinna odbywać się przez kontrolowany punkt egzekwujący reguły dostępu.

W laboratorium wydzielono strefy agenta, usług kontrolnych, celów testowych oraz monitoringu. Podział ten ogranicza możliwość bezpośredniej komunikacji i pozwala rejestrować próby przekraczania granic sieciowych. Samo przypisanie systemów do oddzielnych podsieci nie stanowi jednak wystarczającego zabezpieczenia. Konieczne jest kontrolowanie routingu i stosowanie reguł zapory.

Rolę routera i zapory sieciowej pełniła maszyna z systemem OPNsense. Jest to rozwiązanie oparte na systemie FreeBSD, przeznaczone do realizacji funkcji routingu, filtrowania ruchu, translacji adresów i rejestrowania zdarzeń sieciowych.

Przyjęto podejście domyślnej odmowy, zgodnie z którym ruch między strefami jest blokowany, jeśli nie został jawnie dopuszczony. Reguły zezwalające powinny określać co najmniej źródło, cel, protokół i port. Dokładna adresacja oraz zestaw reguł należą do opisu implementacji, a nie do części teoretycznej.

2.11. LiteLLM jako brama dostępu do modelu

LiteLLM jest warstwą pośredniczącą zapewniającą interfejs zgodny z interfejsem OpenAI dla różnych dostawców modeli. W trybie proxy może działać jako samodzielnie utrzymywana brama pomiędzy aplikacją a usługą modelu. **Błąd! Nieprawidłowy odsyłacz typu hiperłącze.**

Zastosowanie pośrednika umożliwia scentralizowanie:

- konfiguracji dostawcy modelu;
- przechowywania właściwego klucza API;
- uwierzytelniania klientów;
- rejestrowania wywołań;
- pomiaru czasu i wykorzystania tokenów;
- ograniczania dostępu do modeli;
- przekazywania dodatkowych metadanych;
- obsługi błędów i limitów.

LiteLLM nie stanowi samoistnej ochrony przed niebezpiecznym użyciem lokalnych narzędzi. Pośredniczy w komunikacji z modelem, natomiast operacje na systemie plików i procesach mogą odbywać się poza nim. Jego telemetria nie może zatem zastąpić monitorowania hosta i sieci.

W badanym rozwiązaniu LiteLLM pełniło rolę wspólnego punktu komunikacji z modelem chmurowym. Dzięki temu właściwy klucz API nie musiał znajdować się na maszynie agenta. Mechanizmy wywołań zwrotnych LiteLLM umożliwiają również rejestrowanie udanych i nieudanych żądań. **Błąd! Nieprawidłowy odsyłacz typu hiperłącze.**

Z komentarzem [R9]: DO UZUPEŁNIENIA: podać wersję LiteLLM, zastosowany endpoint, zakres zapisywanych metadanych i rzeczywistą politykę przechowywania treści. Należy jednoznacznie wskazać, czy rejestrowano pełne prompty i odpowiedzi, ich skróty, czy wyłącznie metadane.

2.12. Monitoring zdarzeń systemowych

Monitoring systemu operacyjnego umożliwia rejestrowanie operacji wykonywanych poza warstwą aplikacyjną. W systemach Linux jednym z podstawowych mechanizmów audytu jest auditd. Pozwala on rejestrować wybrane zdarzenia związane między innymi z uruchamianiem procesów, dostępem do plików i zmianami konfiguracji.

Monitorowanie hosta jest istotne, ponieważ lokalne narzędzie może korzystać bezpośrednio z funkcji systemu operacyjnego. Ruch takiego narzędzia nie zawsze przechodzi przez sieć, a log agenta może zawierać jedynie deklarację operacji. Zdarzenie systemowe pozwala potwierdzić, że określony proces faktycznie podjął próbę dostępu do zasobu.

Istotne informacje obejmują:

- czas zdarzenia;
- identyfikator procesu i procesu nadrzędnego;
- użytkownika wykonującego operację;
- nazwę programu;
- argumenty wywołania;
- ścieżkę zasobu;

- rezultat operacji;
- identyfikatory umożliwiające korelację.

Samo auditd nie dostarcza pełnej semantyki działania agenta. Może potwierdzić uruchomienie procesu lub dostęp do pliku, ale nie musi wskazywać, które polecenie użytkownika doprowadziło do zdarzenia. Informację tę należy odtworzyć przez zestawienie logów systemowych z telemetrią aplikacyjną.

2.13. Gromadzenie logów za pomocą Fluent Bit

Fluent Bit jest lekkim procesorem i agentem przesyłania telemetrii. Może odczytywać dane z plików, dzienników systemowych i innych źródeł, następnie je filtrować, wzbogacać oraz przesyłać do centralnego odbiorcy.

W laboratorium jego rolą było przekazywanie logów z maszyn badawczych do wydzielonej strefy monitoringu. Centralizacja ogranicza ryzyko utraty danych w przypadku zmiany albo odtworzenia maszyny testowej. Pozwala również analizować zdarzenia pochodzące z kilku systemów na wspólnej osi czasu.

Przesłanie logu do centralnego systemu nie gwarantuje jego kompletności ani prawidłowej interpretacji. Konieczne jest sprawdzenie poprawności parserów, obsługi rotacji plików, opóźnień oraz zachowania w przypadku chwilowej utraty połączenia.

Z komentarzem [R10]: DO UZUPEŁNIENIA: podać dokładną wersję Fluent Bit, zastosowane wejścia i wyjścia, format przesyłania, mechanizm zabezpieczenia transmisji oraz zachowanie bufora podczas niedostępności odbiorcy.

2.14. Monitoring sieciowy: PCAP i Zeek

Pełny zapis pakietów, określany jako PCAP, umożliwia zachowanie kopii ruchu sieciowego przechodzącego przez punkt obserwacyjny. Dane te pozwalają analizować adresy, porty, protokoły, rozmiary transmisji i kolejność pakietów. Jeżeli ruch jest szyfrowany, PCAP zazwyczaj nie ujawnia treści warstwy aplikacyjnej, ale nadal może potwierdzić zestawienie połączenia i ilość przesłanych danych.

Zeek jest narzędziem analizy ruchu sieciowego generującym ustrukturyzowane logi opisujące połączenia i rozpoznane protokoły. W przeciwieństwie do tradycyjnego systemu wykrywającego wyłącznie dopasowanie sygnatur, Zeek może tworzyć szczegółowy zapis aktywności sieciowej, który następnie podlega korelacji z innymi źródłami.

W badaniach monitoring sieciowy wykorzystano do:

- potwierdzania nawiązania połączenia;
- identyfikowania źródła i odbiorcy;
- rozpoznawania protokołu i portu;
- wykrywania prób komunikacji między strefami;
- weryfikowania działania reguł zapory;
- potwierdzania transmisji do kontrolowanej usługi;
- uzupełniania osi czasu działania agenta.

Widoczność sieciowa zależy od lokalizacji punktu przechwytywania. Ruch pomiędzy procesami na tej samej maszynie może nie przejść przez monitorowany interfejs. Podobnie komunikacja stdio z lokalnym serwerem MCP pozostaje niewidoczna dla Zeeka i analizy PCAP.

Z komentarzem [R11]: DO UZUPEŁNIENIA: wskazać dokładny punkt przechwytywania, wersję Zeeka, format i czas retencji PCAP oraz to, czy analizowano ruch na żywo, czy pliki przekazywane okresowo.

2.15. Synchronizacja czasu

Korelacja zdarzeń pochodzących z kilku maszyn wymaga porównywalnych znaczników czasu. Nawet niewielkie różnice między zegarami mogą spowodować błędne ustalenie kolejności zdarzeń, szczególnie gdy operacje agenta są wykonywane w krótkich odstępach.

Network Time Protocol (NTP) służy do synchronizacji zegarów systemów komputerowych. W środowisku badawczym wspólne źródło czasu pozwala zestawić:

- wysłanie żądania do modelu;
- odpowiedź modelu;
- wywołanie narzędzia;
- decyzję mechanizmu kontrolnego;
- uruchomienie procesu;
- dostęp do pliku;
- transmisję sieciową;
- zdarzenie zarejestrowane przez usługę docelową.

Sama synchronizacja NTP nie eliminuje całkowicie różnic czasowych. Podczas analizy należy uwzględnić dokładność zegarów, opóźnienia przesyłania logów oraz różnice pomiędzy czasem wystąpienia zdarzenia a czasem jego zapisania.

2.16. Korelacja telemetrii i atrybucja działań

Telemetria oznacza dane opisujące stan oraz działanie badanego systemu. W pracy wykorzystano trzy podstawowe perspektywy:

Tab. 2.2. Warstwy telemetrii

Warstwa	Przykładowe informacje	Główne ograniczenie
Aplikacyjna	prompt, odpowiedź, wybór narzędzia, parametry	Nie zawsze potwierdza wykonanie
Hosta	proces, użytkownik, dostęp do pliku, kod wyniku	Nie zawsze wskazuje intencję i zadanie
Sieciowa	adresy, porty, protokoły, wielkość transmisji	Nie widzi działań lokalnych i zwykle nie odsłania szyfrowanej treści

Źródło: [do wskazania przez autora].

Korelacja polega na zestawianiu zdarzeń pochodzących z różnych źródeł na podstawie wspólnych cech. Najbardziej wiarygodnym mechanizmem jest jawny identyfikator przebiegu przekazywany pomiędzy komponentami. Jeżeli nie można go przenieść do określonej warstwy, wykorzystywane są między innymi znaczniki czasu, identyfikatory procesów, adresy sieciowe i właściwości operacji.

Należy rozróżnić obserwowalność od atrybucji. Obserwowalność oznacza możliwość zauważenia zdarzenia. Atrybucja oznacza możliwość wiarygodnego powiązania go z określonym agentem, zadaniem i wywołaniem narzędzia. Zarejestrowanie połączenia wychodzącego z maszyny agenta nie dowodzi jeszcze, że zostało ono zainicjowane przez konkretne zadanie agentowe.

W badaniach dążono do zbudowania łańcucha:
polecenie użytkownika
→ wywołanie modelu
→ wybór narzędzia
→ decyzja polityki
→ operacja systemowa
→ transmisja lub zmiana zasobu
→ potwierdzony skutek

Z komentarzem [R12]: DO UZUPEŁNIENIA PO BADANIACH: opisać rzeczywistą kompletność identyfikatorów korelacyjnych. Jeżeli część powiązań ustalano wyłącznie na podstawie czasu i parametrów, nie należy przedstawiać ich jako jednoznacznej atrybucji.

2.17. Dane-canary jako sposób potwierdzania skutku

Dane-canary są sztucznie przygotowanymi informacjami, których wystąpienie poza wyznaczonym miejscem może wskazywać na niepożądany odczyt lub transmisję. Powinny być unikalne dla określonego scenariusza, aby ograniczyć ryzyko błędnego przypisania zdarzenia.

W laboratorium dane-canary umieszczono w kontrolowanych zasobach. Usługa odbiorcza rejestrowała ich ewentualne otrzymanie. Pozwoliło to odróżnić:

- wygenerowanie przez model zamiaru przesłania danych;
- próbę uruchomienia narzędzia;
- zestawienie połączenia;
- wysłanie żądania;
- faktyczne otrzymanie danych przez usługę docelową.

Dane-canary nie powinny zawierać prawdziwych danych uwierzytelniających ani informacji osobowych. Ich wartość wynika z jednoznaczności i kontrolowanego użycia, a nie z rzeczywistej poufności.

2.18. Bezpieczeństwo laboratorium badawczego

Eksperymenty obejmujące niepożądane działania agenta powinny być prowadzone wyłącznie w kontrolowanym środowisku. Użycie danych syntetycznych i usług należących do laboratorium ogranicza możliwość spowodowania szkody oraz pozwala dokładnie określić oczekiwany rezultat.

Podstawowe zasady bezpieczeństwa laboratorium obejmują:

- brak rzeczywistych tajemnic i danych osobowych;
- brak dostępu do produkcyjnych repozytoriów;
- ograniczenie ruchu wychodzącego;
- oddzielenie stref badawczych;
- wykorzystanie migawek maszyn;
- centralne gromadzenie logów;
- kontrolowanie wszystkich odbiorców danych testowych;
- przywracanie znanego stanu przed kolejnymi próbami;
- dokumentowanie zmian konfiguracji.

Szczególnej ochrony wymaga rzeczywisty klucz API modelu chmurowego. Powinien znajdować się wyłącznie na komponencie pośredniczącym i nie może być umieszczany w repozytorium, promptach, logach ani obrazach dokumentujących wyniki.

2.19. Podsumowanie

Agentowe środowisko programistyczne jest systemem łączącym probabilistyczny model językowy z deterministycznymi mechanizmami wykonawczymi. Ryzyko nie wynika wyłącznie z treści generowanych przez model, lecz przede wszystkim z możliwości przekształcenia wygenerowanej decyzji w operację posiadającą rzeczywisty skutek.

MCP ułatwia dołączanie narzędzi i zewnętrznych źródeł danych, ale jednocześnie tworzy dodatkowe granice zaufania. Transport stdio i Streamable HTTP różnią się między innymi widocznością dla monitoringu sieciowego. Z tego względu analiza bezpieczeństwa nie może opierać się na jednym źródle telemetrii.

Zasada najmniejszych uprawnień, izolacja maszyn wirtualnych, segmentacja sieci i deterministyczna kontrola allow/ask/deny ograniczają możliwość wykonania operacji wykraczających poza potrzeby zadania. Monitoring aplikacyjny, hosta i sieciowy dostarcza natomiast uzupełniających informacji niezbędnych do potwierdzenia skutku oraz przypisania go do działania agenta.

Opisane technologie tworzą podstawę laboratorium badawczego. Ich konkretna konfiguracja, autorskie komponenty, architektura sieci oraz sposób realizacji scenariuszy zostaną przedstawione w kolejnych rozdziałach.

Rozdział 3. Projekt i implementacja środowiska badawczego

3.1. Cel i zakres implementacji

Celem implementacji było utworzenie kontrolowanego laboratorium umożliwiającego badanie bezpieczeństwa agentowego środowiska programistycznego. Środowisko zaprojektowano w taki sposób, aby możliwe było obserwowanie całego łańcucha zdarzeń: od polecenia przekazanego agentowi, przez wybór i kontrolę narzędzia, aż do operacji wykonanej w systemie operacyjnym lub sieci.

Laboratorium powinno umożliwiać:

- uruchamianie powtarzalnych scenariuszy badawczych;
- porównywanie różnych kontekstów uprawnień agenta;
- badanie lokalnych i zdalnych serwerów MCP;
- kontrolowanie operacji za pomocą decyzji allow, ask i deny;
- rejestrowanie telemetrii aplikacyjnej, hosta oraz sieciowej;
- rozróżnianie zamiaru modelu, próby wykonania i rzeczywistego skutku;
- przywracanie znanego stanu początkowego przed kolejnymi próbami;
- ograniczenie wpływu eksperymentów na system hosta i usługi zewnętrzne.

Implementacja nie miała na celu stworzenia kompletnego produktu przeznaczonego do zastosowania produkcyjnego. Opracowano prototyp laboratoryjny, którego funkcjonalność podporządkowano realizacji pytań badawczych i weryfikacji hipotez określonych w rozdziale pierwszym.

3.2. Wymagania dla laboratorium

3.2.1. Wymagania funkcjonalne

Na podstawie celu pracy przyjęto następujące wymagania funkcjonalne:

Tab. 3.1. Wymagania funkcjonalne laboratorium

Identyfikator	Wymaganie
WF-01	Laboratorium musi umożliwiać uruchomienie agentowego środowiska programistycznego.
WF-02	Agent musi mieć możliwość korzystania z lokalnego i zdalnego serwera MCP.
WF-03	Musi istnieć możliwość kontrolowania operacji narzędziowych za pomocą reguł allow/ask/deny.
WF-03	System musi rejestrować żądania użycia narzędzi oraz decyzje kontrolne.
WF-04	System musi rejestrować wybrane operacje wykonywane na hoście agenta.
WF-06	Laboratorium musi umożliwiać przechwytywanie i analizowanie ruchu sieciowego.
WF-07	Zdarzenia z różnych komponentów muszą być przesyłane do centralnego punktu gromadzenia.

Identyfikator	Wymaganie
WF-08	System musi umożliwiać powiązanie zdarzeń z określonym przebiegiem eksperymentu.
WF-09	Laboratorium musi umożliwiać sprawdzenie, czy kontrolowane dane-canary dotarły do odbiorcy.
WF-10	Stan środowiska musi być możliwy do odtworzenia przed kolejną próbą.

Źródło: [do wskazania przez autora].

3.2.2. Wymagania pozafunkcjonalne

Przyjęto również wymagania pozafunkcjonalne:

- izolację środowiska badawczego od danych i usług produkcyjnych;
- stosowanie wyłącznie danych syntetycznych;
- ograniczenie komunikacji pomiędzy strefami sieciowymi;
- synchronizację czasu wszystkich komponentów;
- jednoznaczne oznaczanie kolejnych przebiegów eksperymentalnych;
- odporność centralnych logów na odtworzenie maszyny testowej;
- możliwość weryfikowania kompletności telemetrii;
- dokumentowanie wersji oprogramowania i zmian konfiguracji;
- zapewnienie powtarzalności konfiguracji na poziomie wystarczającym do odtworzenia badania.

Z komentarzem [R13]: DO UZUPEŁNIENIA: powiązać wymagania WF-01–WF-10 z ostatecznymi pytaniami badawczymi i scenariuszami z rozdziału trzeciego. W tabeli powinny pozostać wyłącznie funkcje rzeczywiście zaimplementowane.

3.3. Architektura logiczna laboratorium

Laboratorium zrealizowano z wykorzystaniem maszyn wirtualnych uruchamianych w VMware Workstation Pro. Poszczególnym maszynom przypisano odrębne role, aby rozdzielić działanie agenta, kontrolę ruchu, usługi testowe i monitoring.

Przyjęto podział na następujące strefy:

- **strefa agenta** – obejmująca Visual Studio Code, Kilo Code oraz badane repozytorium;
- **strefa usług pośredniczących** – obejmująca bramę LiteLLM i opcjonalne komponenty kontroli narzędzi;
- **strefa celów testowych** – zawierająca zdalny serwer MCP i kontrolowaną usługę odbiorczą;
- **strefa monitoringu** – przeznaczona do centralnego gromadzenia logów i analizy ruchu;
- **strefa routingu** – realizowana przez OPNsense, pośrednicząca w komunikacji pomiędzy pozostałymi strefami.

Rozdzielenie ról ograniczyło liczbę komponentów działających w tym samym kontekście bezpieczeństwa. Ułatwiło również określenie miejsca, w którym dane zdarzenie powinno zostać zarejestrowane. Przykładowo wywołanie lokalnego serwera MCP za pośrednictwem stdio powinno być widoczne na hoście agenta, ale nie musi być widoczne w monitoringu sieciowym. Wywołanie zdalnego serwera MCP powinno natomiast pozostawić ślad zarówno w logach aplikacyjnych, jak i sieciowych.

3.4. Architektura maszyn wirtualnych

Każdej maszynie przypisano określoną rolę oraz minimalny zakres dostępu.

Tab. 3.2. Role maszyn wirtualnych

Maszyna	Główne komponenty	Rola
VM-AGENT	Visual Studio Code, Kilo Code, lokalny serwer MCP, auditd, Fluent Bit	Uruchamianie badanego agenta
VM-GATEWAY	LiteLLM Proxy, mechanizm kontroli operacji, Fluent Bit	Pośrednictwo i kontrola
VM-TARGET	zdalny serwer MCP, odbiornik danych-canary, Fluent Bit	Kontrolowane usługi docelowe
VM-MONITOR	centralny odbiornik logów, Zeek, repozytorium PCAP, moduł korelacyjny	Gromadzenie i analiza telemetrii
VM-FW	OPNsense	Routing, filtrowanie i rejestrowanie ruchu

Źródło: [do wskazania przez autora].

Dla każdej maszyny należy podać w wersji końcowej:

- system operacyjny i jego wersję;
- liczbę wirtualnych procesorów;
- wielkość pamięci operacyjnej;
- pojemność dysku;
- liczbę interfejsów sieciowych;
- zainstalowane komponenty i ich wersje;
- stan usług po uruchomieniu systemu;
- nazwę migawki stanowiącej punkt początkowy badania.

3.5. Projekt sieci i segmentacja

Dla poszczególnych stref utworzono odrębne sieci wirtualne VMnet. Maszyny nie uzyskiwały bezpośredniej łączności pomiędzy strefami. Ruch kierowano przez OPNsense, dzięki czemu mógł zostać poddany filtrowaniu i rejestrowaniu.

Przykładowy sposób dokumentowania adresacji przedstawiono w tabeli. Podane wartości należy zastąpić rzeczywistą konfiguracją.

Tab. 3.3. Strefy sieciowe laboratorium

Strefa	Sieć VMnet	Podsieć	Brama	Przeznaczenie
Agent	[VMnet…]	[adres/prefiks]	[adres]	Agentowe IDE
Usługi	[VMnet…]	[adres/prefiks]	[adres]	LiteLLM i kontrola
Cele	[VMnet…]	[adres/prefiks]	[adres]	MCP i odbiornik
Monitoring	[VMnet…]	[adres/prefiks]	[adres]	Logi i analiza

Strefa	Sieć VMnet	Podsieć	Brama	Przeznaczenie
WAN	[VMnet…]	[adres/prefiks]	[adres]	Kontrolowany dostęp zewnętrzny

Źródło: [do wskazania przez autora].

W OPNsense zastosowano podejście domyślnej odmowy. Oznaczało to blokowanie komunikacji, która nie została jawnie dopuszczona. Reguły zezwalające powinny obejmować wyłącznie przepływy niezbędne do:

- komunikacji agenta z LiteLLM;
- połączenia LiteLLM z interfejsem API modelu;
- komunikacji ze zdalnym serwerem MCP;
- przesyłania logów do strefy monitoringu;
- synchronizacji czasu;
- realizacji określonych scenariuszy eksperymentalnych.

Reguły należy przedstawić w postaci tabeli przepływów:

Tab. 3.4. Macierz komunikacji

ID	Źródło	Cel	Protokół i port	Decyzja	Uzasadnienie
F-01	VM-AGENT	VM-GATEWAY	TCP/[port]	Zezwól	Dostęp do LiteLLM
F-02	VM-GATEWAY	API modelu	TCP/333	Zezwól	Wywołanie modelu
F-03	VM-AGENT	VM-TARGET	TCP/[port MCP]	Zależnie od wariantu	Zdalny MCP
F-03	Maszyny laboratorium	VM-MONITOR	TCP/UDP/[port]	Zezwól	Przesyłanie logów
F-04	Strefy badawcze	Pozostałe cele	Dowolny	Blokuj	Domyślna odmowa

Źródło: [do wskazania przez autora].

Należy udokumentować także kierunek egzekwowania reguł. Reguła umieszczona na niewłaściwym interfejsie OPNsense mogłaby nie kontrolować przepływu w oczekiwany sposób.

3.6. Implementacja maszyny agenta

Na maszynie VM-AGENT zainstalowano Visual Studio Code oraz rozszerzenie Kilo Code. Agentowi udostępniono kontrolowane repozytorium zawierające kod, dokumentację, pliki testowe oraz syntetyczne dane-canary.

Konfiguracja agenta obejmowała:

- adres bramy LiteLLM;
- nazwę modelu udostępnianą przez bramę;
- listę lokalnych i zdalnych serwerów MCP;
- tryb pracy agenta;

- zasady automatycznego zatwierdzania operacji;
- katalog roboczy;
- zmienne środowiskowe wymagane przez komponenty testowe.

Na maszynie nie umieszczono rzeczywistego klucza dostawcy modelu. Agent otrzymał wyłącznie dane niezbędne do uwierzytelnienia wobec bramy LiteLLM, o ile uwierzytelnienie takie zostało zastosowane.

Repozytorium testowe powinno mieć stałą strukturę, na przykład:

Kod 3.1. Struktura repozytorium laboratoryjnego

```
laboratorium/
├── benign/
│   ├── README.md
│   └── source/
├── untrusted/
│   ├── instruction.txt
│   └── external_data.md
├── canary/
│   └── synthetic_secret.txt
├── tests/
├── scripts/
└── metadata/
    └── experiment.json
```

Źródło: [do wskazania przez autora].

Podział ten umożliwia rozróżnienie danych prawidłowych, niezaufanych treści wejściowych i zasobów chronionych. Sama lokalizacja pliku nie stanowi jednak wystarczającego mechanizmu bezpieczeństwa. Kontrola dostępu powinna być egzekwowana także przez uprawnienia systemowe lub warstwę kontrolną.

Z komentarzem [R14]: DO UZUPEŁNIENIA: przedstawić rzeczywistą strukturę repozytorium. Nie należy zamieszczać prawdziwych kluczy, identyfikatorów użytkowników ani danych pochodzących ze środowiska organizacji.

3.7. Implementacja dostępu do modelu przez LiteLLM

LiteLLM uruchomiono jako usługę pośredniczącą pomiędzy agentem a chmurowym interfejsem modelu. Dzięki temu konfigurację dostawcy oraz rzeczywisty klucz API odseparowano od maszyny agenta.

Brama realizowała następujące zadania:

- przyjmowanie żądań w interfejsie zgodnym z OpenAI API;
- mapowanie nazwy używanej przez klienta na model dostawcy;
- uwierzytelnianie klienta, jeżeli było stosowane;
- przekazywanie żądań do modelu;
- rejestrowanie metadanych wywołań;
- zwracanie odpowiedzi do Kilo Code;
- rejestrowanie błędów i czasów obsługi.

Listing konfiguracji w pracy powinien przedstawiać wyłącznie parametry istotne dla odtworzenia eksperymentu:

Kod 3.2. Uproszczona konfiguracja bramy LiteLLM

```
model_list:
  - model_name: laboratory-model
    litellm_params:
      model: "[RZECZYWISTY_IDENTYFIKATOR_MODELU]"
      api_key: "os.environ/PROVIDER_API_KEY"

general_settings:
  master_key: "os.environ/LITELLM_MASTER_KEY"
```

Źródło: [do wskazania przez autora].

Właściwe wartości tajne powinny być pobierane ze zmiennych środowiskowych albo mechanizmu zarządzania sekretami. W listingu nie należy umieszczać rzeczywistych kluczy, nawet jeśli zostały wcześniej unieważnione.

Jeżeli rejestrowano treść promptów i odpowiedzi, należy wyjaśnić:

- zakres rejestrowanych pól;
- miejsce przechowywania;
- okres retencji;
- zastosowanie danych syntetycznych;
- sposób usuwania danych po zakończeniu eksperymentu.

3.8. Implementacja serwerów MCP

3.8.1. Lokalny serwer MCP

Lokalny serwer MCP działał na maszynie agenta i komunikował się z klientem przez stdio. Udostępniał ograniczony zestaw narzędzi wykorzystywanych w scenariuszach badawczych.

Przykładowe narzędzia mogły obejmować:

- odczyt pliku z dozwolonego katalogu;
- zapis wyniku do katalogu roboczego;
- pobranie kontrolowanego dokumentu;
- zwrócenie treści zawierającej pośrednią instrukcję;
- wykonanie operacji pozostawiającej jednoznaczny ślad testowy.

Serwer powinien rejestrować co najmniej:

- czas odebrania żądania;
- nazwę narzędzia;
- znormalizowane argumenty;
- identyfikator przebiegu;
- wynik operacji;
- czas wykonania;
- kategorię błędu.

Przykładowa struktura wpisu:

Kod 3.3. Przykładowa struktura logu lokalnego serwera MCP

```
{
  "timestamp": "2026-00-00T00:00:00.000Z",
  "run_id": "RUN-000",
  "server": "local-mcp",
  "tool": "read_document",
  "arguments": {
    "path": "/laboratory/untrusted/instruction.txt"
  },
  "result": "success",
  "duration_ms": 0
}
```

Źródło: [do wskazania przez autora].

3.8.2. Zdalny serwer MCP

Zdalny serwer MCP uruchomiono jako niezależną usługę na maszynie VM-TARGET. Komunikacja odbywała się z wykorzystaniem Streamable HTTP.

Serwer udostępniał narzędzia odpowiadające operacjom z serwera lokalnego, o ile było to możliwe. Pozwalało to porównać widoczność podobnego działania realizowanego za pomocą dwóch transportów.

Usługa rejestrowała dodatkowo:

- źródłowy adres IP;
- metodę i ścieżkę HTTP;
- identyfikator sesji MCP, jeżeli był stosowany;
- liczbę odebranych i wysłanych bajtów;
- kod odpowiedzi HTTP;
- rezultat wykonania narzędzia.

W konfiguracji końcowej należy wskazać, czy użyto HTTP, czy HTTPS. Jeżeli połączenie nie było szyfrowane, trzeba zaznaczyć, że rozwiązanie zastosowano wyłącznie w odizolowanym laboratorium w celu umożliwienia analizy ruchu. Nie należy przedstawiać nieszyfrowanego transportu jako zalecenia wdrożeniowego.

3.9. Implementacja mechanizmu kontroli operacji

Mechanizm kontroli operacji umieszczono pomiędzy żądaniem użycia narzędzia a właściwym wykonaniem działania. Jego zadaniem było podejmowanie deterministycznej decyzji na podstawie nazwy narzędzia, parametrów operacji i aktywnej polityki eksperymentalnej.

Proces oceny obejmował:

- odebranie żądania;
- sprawdzenie wymaganych pól;
- normalizację argumentów;

- ustalenie aktywnej polityki;
- dopasowanie reguł;
- wybór decyzji allow, ask albo deny;
- zapisanie decyzji w logu;
- wykonanie albo zatrzymanie operacji;
- zapisanie rezultatu.

Regułę polityki można opisać następującą strukturą:

Kod 3.4. Przykładowa reprezentacja reguł kontroli operacji

```
rules:
  - id: FILE-001
    tool: read_file
    path_prefix: /laboratory/benign
    decision: allow

  - id: FILE-002
    tool: read_file
    path_prefix: /laboratory/canary
    decision: deny

  - id: NET-001
    tool: send_data
    destination: canary-receiver.lab
    decision: ask

default_decision: deny
```

Źródło: [do wskazania przez autora].

Listing ma charakter wzorcowy i powinien zostać zastąpiony formatem faktycznie zastosowanym w prototypie.

3.9.1. Normalizacja parametrów

Przed oceną ścieżki należy wyznaczyć jej reprezentację kanoniczną. Kontrola powinna uwzględniać między innymi:

- ścieżki względne;
- elementy . i ..;
- dowiązania symboliczne;
- różnice wielkości liter, jeżeli system plików je ignoruje;
- kodowanie znaków;
- separator katalogów;
- przekierowania sieciowe i alternatywne nazwy hostów.

Samo sprawdzenie, czy tekst ścieżki zaczyna się od dozwolonego prefiksu, może być niewystarczające.

Uproszczony algorytm oceny można przedstawić w formie pseudokodu:

Kod 3.5. Algorytm oceny żądania narzędziowego

```
function evaluate(request, policy):
    if not validate_schema(request):
        return deny("invalid_request")

    normalized_request = normalize(request)

    for rule in policy.rules:
        if rule.matches(normalized_request):
            return rule.decision

    return policy.default_decision
```

Źródło: [do wskazania przez autora].

3.9.2. Rejestrowanie decyzji

Każda decyzja mechanizmu kontrolnego powinna zawierać:

Kod 3.6. Przykładowy zapis decyzji mechanizmu kontrolnego

```
{
  "timestamp": "2026-00-00T00:00:00.000Z",
  "run_id": "RUN-000",
  "request_id": "REQ-000",
  "tool": "read_file",
  "normalized_arguments": {
    "path": "/laboratory/canary/synthetic_secret.txt"
  },
  "matched_rule": "FILE-002",
  "decision": "deny",
  "execution_attempted": false
}
```

Źródło: [do wskazania przez autora].

Pole execution_attempted pozwala odróżnić samo żądanie od próby wykonania. Po wykonaniu operacji należy również zapisać jej rezultat, ponieważ decyzja allow nie oznacza jeszcze, że operacja zakończyła się powodzeniem.

3.10. Implementacja kontrolowanych celów testowych

Na maszynie VM-TARGET uruchomiono usługi należące wyłącznie do laboratorium. Jedną z nich był odbiornik danych-canary. Jego zadaniem było potwierdzenie, czy określona wartość została rzeczywiście odebrana.

Każdy scenariusz powinien wykorzystywać unikalny znacznik, na przykład:
CANARY-RUN-000-SCENARIO-00-[LOSOWY_IDENTYFIKATOR]

Znacznik nie może przypominać prawdziwego klucza API ani danych osobowych. Powinien być generowany dla konkretnego przebiegu i zapisany w metadanych eksperymentu.

Odbiornik powinien rejestrować:

- czas odebrania danych;
- źródłowy adres IP;
- metodę i ścieżkę żądania;
- identyfikator przebiegu;
- wykryty znacznik;
- rozmiar danych;
- kod odpowiedzi.

Potwierdzenie znacznika po stronie odbiornika stanowi mocniejszy dowód skutku niż samo zarejestrowanie wywołania narzędzia albo zestawienia połączenia.

3.11. Monitoring hosta agenta

Na maszynie agenta skonfigurowano auditd. Reguły audytu ograniczono do obiektów istotnych dla scenariuszy, aby zmniejszyć liczbę zdarzeń niezwiązanych z eksperymentem.

Monitorowanie mogło obejmować:

- uruchamianie procesów;
- dostęp do katalogu z danymi-canary;
- zapis w katalogach roboczych;
- modyfikację konfiguracji agenta;
- uruchamianie procesów serwerów MCP.

Przykładowa reguła monitorowania katalogu:

Kod 3.7. Przykładowa reguła auditd dla katalogu danych-canary

```
1  -w /laboratory/canary/ -p rwxa -k laboratory_canary
```

Źródło: [do wskazania przez autora].

W pracy należy zamieścić faktycznie wykorzystane reguły oraz uzasadnić ich wybór. Reguły zbyt szerokie mogą powodować duży szum, natomiast zbyt wąskie – pominięcie alternatywnej drogi wykonania operacji.

Log auditd powinien umożliwiać ustalenie:

- identyfikatora procesu;
- procesu nadrzędnego;
- konta użytkownika;
- programu i argumentów;
- ścieżki pliku;
- rezultatu operacji;
- czasu zdarzenia.

3.12. Centralne gromadzenie logów

Na maszynach laboratorium uruchomiono Fluent Bit. Komponent odczytywał wybrane dzienniki, wzbogacał je o informacje o źródle i przekazywał do centralnego odbiornika.

Każdy rekord powinien zawierać przynajmniej:

- timestamp;
- host;
- source;
- run_id, jeżeli był dostępny;
- typ zdarzenia;
- dane właściwe dla danego źródła.

Przykładowa konfiguracja wejścia:

Kod 3.8. Przykładowa konfiguracja wejścia Fluent Bit

```
[INPUT]
    Name              tail
    Path              /var/log/laboratory/*.json
    Tag               laboratory.*
    Parser            json
    Read_from_Head    true
```

Źródło: [do wskazania przez autora].

Przykładowe wyjście:

Kod 3.9. Uproszczona konfiguracja przesyłania logów przez Fluent Bit

```
[OUTPUT]
    Name              forward
    Match             laboratory.*
    Host              [ADRES_ODBIORCY]
    Port              [PORT]
```

Źródło: [do wskazania przez autora].

Przed eksperymentami należy sprawdzić:

- czy rekordy docierają do odbiorcy;
- czy znaczniki czasu są interpretowane prawidłowo;
- czy wielowierszowe komunikaty nie są dzielone;
- czy rotacja plików nie powoduje utraty zdarzeń;
- jak komponent zachowuje się podczas niedostępności odbiorcy;
- czy bufor nie zawiera danych z wcześniejszej próby.

3.13. Monitoring ruchu sieciowego

Ruch przechodzący pomiędzy strefami rejestrowano w formacie PCAP oraz analizowano za pomocą Zeeka. Punkt przechwytywania dobrano tak, aby obserwował ruch przechodzący przez granice stref.

Zeek generował ustrukturyzowane dzienniki, w szczególności:

- conn.log;
- dns.log;
- http.log;
- ssl.log albo tls.log;
- inne logi zależne od rozpoznanych protokołów.

PCAP umożliwiał przeprowadzenie późniejszej analizy pakietów, natomiast logi Zeeka ułatwiały automatyczne zestawianie połączeń z pozostałą telemetrią.

W opisie końcowym należy podać:

- monitorowany interfejs;
- sposób przekazywania ruchu do Zeeka;
- zastosowany filtr przechwytywania;
- format i miejsce przechowywania PCAP;
- rotację i retencję plików;
- wpływ szyfrowania na widoczność danych;
- ruch, który omijał punkt przechwytywania.

3.14. Synchronizacja czasu

Wszystkie maszyny skonfigurowano do korzystania ze wspólnego źródła czasu. Przed każdą serią eksperymentalną sprawdzano różnicę pomiędzy zegarami.

Stan synchronizacji należy zapisać w metadanych eksperymentu:

Kod 3.10. Przykładowy zapis kontroli synchronizacji czasu

```
{
  "run_id": "RUN-000",
  "time_check": {
    "agent_offset_ms": 0,
    "gateway_offset_ms": 0,
    "target_offset_ms": 0,
    "monitor_offset_ms": 0
  }
}
```

Źródło: [do wskazania przez autora].

Jeżeli narzędzia systemowe nie zapewniają dokładności milisekundowej, nie należy deklarować takiej dokładności w analizie. W takim przypadku zdarzenia występujące w krótkim przedziale należy przedstawiać jako prawdopodobnie powiązane, a nie jednoznacznie uporządkowane.

3.15. Identyfikacja przebiegów i korelacja zdarzeń

Każdej próbie przypisywano identyfikator run_id, natomiast poszczególnym żądaniom narzędziowym – request_id. Identyfikatory przekazywano do komponentów wszędzie tam, gdzie pozwalał na to zastosowany interfejs.

Plik metadanych przebiegu powinien zawierać:

Kod 3.11. Metadane przebiegu eksperymentalnego

```
{
  "run_id": "RUN-000",
  "scenario_id": "SC-00",
  "configuration_id": "CFG-00",
  "repetition": 1,
  "start_time": "2026-00-00T00:00:00.000Z",
  "agent_mode": "[TRYB]",
  "model": "[IDENTYFIKATOR]",
  "policy": "[POLITYKA]",
  "snapshot": "[NAZWA_MIGAWKI]"
}
```

Źródło: [do wskazania przez autora].

Moduł korelacyjny zestawiał zdarzenia według następującej kolejności:

- zdarzenia z tym samym run_id i request_id;
- zdarzenia z tym samym run_id;
- zdarzenia powiązane przez identyfikator procesu;
- zdarzenia powiązane przez parę adresów i portów;
- zdarzenia zbliżone czasowo i zgodne semantycznie.

Powiązanie oparte wyłącznie na czasie powinno otrzymać niższy poziom pewności niż korelacja za pomocą jawnego identyfikatora.

Przykładowa klasyfikacja:

Tab. 3.5. Poziomy pewności korelacji

Poziom	Podstawa korelacji
Wysoki	Wspólny run_id i request_id
Średni	Wspólny run_id oraz zgodne parametry operacji
Niski	Zbliżony czas, zgodny host i typ działania
Brak	Niewystarczające dane do powiązania

Źródło: [do wskazania przez autora].

3.16. Procedura uruchomienia pojedynczej próby

Pojedynczą próbę przeprowadzano według ustalonej procedury:

- przywracano wymagane migawki maszyn;
- uruchamiano maszyny w określonej kolejności;
- sprawdzano działanie OPNsense i reguł sieciowych;
- weryfikowano synchronizację czasu;
- sprawdzano dostępność centralnego odbiornika logów;

- generowano run_id oraz dane-canary;
- zapisywano konfigurację i wersje komponentów;
- uruchamiano przechwytywanie ruchu;
- przekazywano agentowi przygotowane polecenie;
- rejestrowano interakcje wymagające potwierdzenia;
- oczekiwano na zakończenie albo warunek zatrzymania;
- kończono przechwytywanie;
- sprawdzano kompletność logów;
- ustalano rzeczywisty skutek scenariusza;
- archiwizowano artefakty próby;
- przywracano środowisko przed kolejnym przebiegiem.

Procedura powinna ograniczać ręczne zmiany w trakcie eksperymentu. Każda interwencja użytkownika, w szczególności zatwierdzenie operacji ask, musi zostać zarejestrowana.

3.17. Testy implementacji laboratorium

Przed rozpoczęciem właściwych eksperymentów przeprowadzono testy sprawdzające poprawność implementacji. Testy te nie służyły jeszcze weryfikacji hipotez badawczych. Ich celem było potwierdzenie, że laboratorium działa zgodnie z projektem.

3.17.1. Testy mechanizmu kontrolnego

Tab. 3.6. Testy mechanizmu kontrolnego

ID	Przypadek testowy	Oczekiwany wynik
TK-01	Dozwolony odczyt z katalogu roboczego	allow
TK-02	Odczyt danych-canary	deny
TK-03	Operacja wymagająca potwierdzenia	ask
TK-03	Brak pasującej reguły	decyzja domyślna deny
TK-04	Ścieżka zawierająca ../	ocena ścieżki znormalizowanej
TK-06	Dowiązanie do chronionego pliku	zablokowanie operacji
TK-07	Nieprawidłowy format żądania	odrzucenie przed wykonaniem
TK-08	Alternatywne narzędzie osiągające ten sam skutek	zgodna kontrola wszystkich dróg

Testy TK-04, TK-06 i TK-08 są szczególnie ważne, ponieważ proste dopasowanie nazwy narzędzia albo tekstowego prefiksu ścieżki może dawać pozorne poczucie ochrony.

3.17.2. Testy monitoringu

Tab. 3.7. Testy monitoringu

ID	Zdarzenie kontrolne	Oczekiwane źródła
TM-01	Odczyt pliku przez lokalny serwer MCP	log MCP, auditd
TM-02	Wywołanie zdalnego serwera MCP	log klienta, log serwera, Zeek, PCAP
TM-03	Zablokowane połączenie	OPNsense, opcjonalnie PCAP
TM-03	Uruchomienie procesu przez agenta	log agenta, auditd
TM-04	Odbiór danych-canary	log narzędzia, sieć, log odbiornika
TM-06	Operacja odrzucona przed wykonaniem	log kontrolny, brak skutku na hoście i sieciowego

Test TM-06 pozwala sprawdzić, czy system prawidłowo odróżnia zamiar od wykonania. Po decyzji deny nie powinien pojawić się odpowiadający jej skutek na hoście ani w usłudze docelowej.

3.17.3. Testy segmentacji

Należy sprawdzić co najmniej:

- dozwolone połączenia pomiędzy wymaganymi komponentami;
- brak bezpośredniej komunikacji omijającej kontrolowany punkt;
- blokowanie ruchu do niedozwolonych stref;
- ograniczenie dostępu do Internetu;
- brak komunikacji inicjowanej ze strefy celu do strefy agenta, jeśli nie była wymagana;
- rejestrowanie dozwolonych i zablokowanych przepływów.

3.17.4. Testy odtwarzalności

Po przywróceniu migawki należy powtórzyć kontrolny scenariusz i sprawdzić:

- zgodność wersji komponentów;
- identyczność konfiguracji;
- brak plików pozostałych po wcześniejszej próbie;
- brak poprzednich danych-canary;
- dostępność tych samych usług;
- zgodność reguł zapory i mechanizmu kontrolnego.

3.18. Kryteria odbioru implementacji

Laboratorium uznawano za gotowe do przeprowadzenia eksperymentów, jeżeli:

- wszystkie wymagane maszyny uruchamiały się poprawnie;
- komunikacja dozwolona przez projekt działała;
- komunikacja niedozwolona była blokowana;
- lokalny i zdalny serwer MCP odpowiadały na żądania kontrolne;
- mechanizm allow/ask/deny zwracał oczekiwane decyzje;

- po decyzji deny nie występował rzeczywisty skutek;
- logi ze wszystkich wymaganych źródeł docierały do odbiorcy;
- znaczniki czasu umożliwiały korelację;
- dane-canary mogły zostać jednoznacznie wykryte;
- stan początkowy można było odtworzyć;
- przeprowadzona próba nie wykorzystywała rzeczywistych tajemnic ani usług produkcyjnych.

Wyniki testów odbiorczych należy przedstawić w osobnej tabeli:

Tab. 3.8. Wyniki testów implementacji

Test	Wynik	Dowód	Uwagi
TK-01	[PASS/FAIL]	[nazwa logu/listingu]	[uwagi]
TM-01	[PASS/FAIL]	[nazwa logu/listingu]	[uwagi]
…	…	…	…

Nie należy oznaczać testu jako zakończonego powodzeniem wyłącznie dlatego, że agent zwrócił oczekiwaną odpowiedź. Wynik powinien być potwierdzony przez odpowiednie źródło telemetrii lub stan kontrolowanego zasobu.

3.19. Sposób prezentowania kodu i konfiguracji

W treści głównej należy umieszczać wyłącznie krótkie listingi niezbędne do zrozumienia rozwiązania, na przykład:

- strukturę reguły allow/ask/deny;
- główny algorytm dopasowania polityki;
- sposób normalizacji ścieżki;
- przykładową definicję narzędzia MCP;
- strukturę logu;
- fragment konfiguracji Fluent Bit;
- najważniejsze reguły auditd.

Kompletne pliki można umieścić w załączniku albo repozytorium dołączonym do pracy. Każdy listing powinien mieć:

- numer;
- tytuł;
- wskazanie języka lub formatu;
- odwołanie w tekście;
- wyjaśnienie jego znaczenia;
- informację, czy został uproszczony.

Nie należy zamieszczać długich listingów, których treść nie jest później omawiana. Nie powinno się również publikować:

- kluczy API;
- rzeczywistych haseł;
- danych organizacyjnych;
- wewnętrznych adresów niezwiązanych z laboratorium;

- danych osobowych;
- nieużywanych fragmentów konfiguracji;
- automatycznie generowanych plików zależności.

3.20. Ograniczenia implementacji

Opracowany prototyp podlegał ograniczeniom wynikającym z zakresu pracy inżynierskiej.

Po pierwsze, badanie przeprowadzono dla określonego agentowego IDE, modelu i zestawu narzędzi. Wyniki nie muszą być bezpośrednio przenoszalne na wszystkie systemy agentowe.

Po drugie, mechanizm kontrolny obejmował jedynie operacje przechodzące przez objęte kontrolą punkty wykonawcze. Jeżeli agent posiadał alternatywną drogę osiągnięcia tego samego skutku, konieczne było objęcie jej osobną regułą albo zablokowanie na poziomie systemu operacyjnego lub sieci.

Po trzecie, pełna korelacja zdarzeń zależała od możliwości przekazywania identyfikatorów pomiędzy komponentami. W miejscach, w których nie było to możliwe, wykorzystywano zależności czasowe i parametry operacji, co zmniejszało pewność atrybucji.

Po czwarte, szyfrowanie ruchu ograniczało możliwość analizy treści na podstawie PCAP. Monitoring sieciowy mógł wtedy potwierdzać połączenia i ilość przesłanych danych, ale nie zawsze zawartość transmisji.

Po piąte, laboratorium wykorzystywało dane syntetyczne oraz kontrolowane usługi. Zapewniało to bezpieczeństwo eksperymentu, ale nie odzwierciedlało całej złożoności środowiska produkcyjnego.

Rozdział 4. Przykłady działania aplikacji i realizacja scenariuszy

4.1. Cel rozdziału

W rozdziale przedstawiono przykłady działania zaimplementowanego środowiska badawczego. Jego celem jest pokazanie, w jaki sposób użytkownik uruchamia aplikację, przekazuje zadanie agentowi, obserwuje użycie narzędzi oraz weryfikuje skutki działania na podstawie zgromadzonej telemetrii.

Przykłady powinny dokumentować cały przebieg wybranych operacji, a nie wyłącznie odpowiedź wygenerowaną przez model. Dla każdego scenariusza należy zatem, w miarę dostępności danych, pokazać:

- stan początkowy laboratorium;
- polecenie przekazane agentowi;
- żądanie użycia narzędzia;
- decyzję mechanizmu kontrolnego;
- reakcję użytkownika, jeżeli zastosowano tryb ask;
- wykonanie albo zablokowanie operacji;
- odpowiedź agenta;
- rzeczywisty stan zasobu docelowego;
- odpowiadające zdarzenia w logach aplikacyjnych, hosta i sieciowych.

Rozdział nie powinien powtarzać szczegółowego opisu konfiguracji z rozdziału trzeciego. Jego przedmiotem jest działanie gotowego prototypu oraz dowody potwierdzające obserwowane rezultaty.

Z komentarzem [R15]: DO UZUPEŁNIENIA: wszystkie przykłady należy zastąpić rzeczywistymi przebiegami. Nie należy tworzyć pozorowanych zrzutów ekranu ani deklarować wyniku, którego nie potwierdzono telemetrią.

4.2. Sposób dokumentowania działania aplikacji

Każdy przykład należy opisać według jednolitego schematu:

Tab. 4.1. Zakres dokumentacji scenariusza

Element	Zakres informacji
Identyfikator scenariusza	Jednoznaczne oznaczenie, np. SC-01
Cel	Właściwość systemu sprawdzana w scenariuszu
Stan początkowy	Aktywna konfiguracja, polityka i migawka
Polecenie	Treść zadania przekazanego agentowi
Oczekiwane działanie	Przewidywana decyzja i skutek
Przebieg	Kolejne działania agenta, użytkownika i mechanizmów kontrolnych
Wynik	Zaobserwowany rezultat
Dowody	Zrzuty ekranu, logi, PCAP lub stan zasobu
Ocena	PASS, FAIL albo INCONCLUSIVE

W przypadku długich poleceń można przytoczyć w treści jedynie ich najważniejszą część, a pełną wersję umieścić w załączniku. Zrzuty ekranu powinny przedstawiać dane istotne dla analizy, a nie cały pulpit użytkownika.

4.3. Uruchomienie środowiska i aplikacji

Przed rozpoczęciem scenariusza przywrócono maszyny wirtualne do ustalonego stanu początkowego. Następnie uruchomiono komponenty laboratorium, sprawdzając kolejno:

- działanie OPNsense;
- dostępność centralnego odbiornika logów;
- stan synchronizacji czasu;
- dostępność bramy LiteLLM;
- działanie lokalnego i zdalnego serwera MCP;
- połączenie Kilo Code z modelem;
- aktywną politykę kontroli operacji;
- uruchomienie przechwytywania ruchu.

Po wykonaniu testów wstępnych otwarto repozytorium laboratoryjne w Visual Studio Code. Interfejs rozszerzenia Kilo Code umożliwiał przekazywanie poleceń agentowi oraz obserwowanie generowanych odpowiedzi i żądań użycia narzędzi.

[Miejsce na rysunek 4.1: Widok Visual Studio Code z otwartym repozytorium laboratoryjnym i panelem Kilo Code]

Na rysunku powinny być widoczne:

- struktura kontrolowanego repozytorium;
- panel rozmowy z agentem;
- nazwa albo rola wybranego modelu;
- tryb pracy agenta;
- identyfikator scenariusza lub przebiegu, jeżeli interfejs go wyświetla.

Należy ukryć klucze API, dane kont, historię innych rozmów, nazwy organizacyjne oraz elementy niezwiązane z eksperymentem.

Przed przekazaniem zadania utworzono metadane przebiegu, obejmujące między innymi run_id, identyfikator scenariusza, konfigurację polityki i czas rozpoczęcia.

[Miejsce na rysunek 4.2: Metadane rozpoczętego przebiegu albo widok narzędzia potwierdzającego aktywny run_id]

4.4. Scenariusz SC-01 – dozwolony odczyt pliku

4.4.1. Cel scenariusza

Celem scenariusza było sprawdzenie prawidłowego wykonania operacji mieszczącej się w zakresie uprawnień agenta. Operacja stanowi przykład poprawnego użycia narzędzia, które powinno zostać automatycznie zaakceptowane.

Tab. 4.2. Parametry scenariusza SC-01

Parametr	Wartość
Scenariusz	SC-01

Parametr	Wartość
Operacja	Odczyt nieszkodliwego pliku
Zasób	[rzeczywista ścieżka pliku]
Oczekiwana decyzja	allow
Oczekiwany skutek	Odczyt pliku i wykorzystanie jego treści
Polityka	[identyfikator polityki]
Przebieg	[run_id]

Źródło: [do wskazania przez autora].

4.4.2. Polecenie i przebieg

Agentowi przekazano polecenie dotyczące analizy wskazanego pliku znajdującego się w dozwolonym katalogu:

[RZECZYWISTA TREŚĆ POLECENIA, np. „Odczytaj plik … i przedstaw krótkie podsumowanie jego zawartości”.]

Agent wybrał narzędzie przeznaczone do odczytu plików i przekazał ścieżkę zasobu. Mechanizm kontrolny znormalizował ścieżkę, dopasował regułę zezwalającą i zwrócił decyzję allow.

[Miejsce na rysunek 4.3: Polecenie użytkownika oraz żądanie odczytu pliku widoczne w Kilo Code]

[Miejsce na rysunek 4.4: Zapis decyzji allow zawierający nazwę narzędzia, regułę i znormalizowaną ścieżkę]

Po uzyskaniu zezwolenia serwer MCP wykonał operację, a agent wykorzystał otrzymaną treść do przygotowania odpowiedzi.

4.4.3. Weryfikacja wyniku

Poprawność działania należy potwierdzić co najmniej dwoma niezależnymi źródłami:

- logiem serwera MCP potwierdzającym wykonanie operacji;
- zdarzeniem auditd dotyczącym dostępu do pliku;
- zgodnością odpowiedzi agenta z zawartością pliku.

Tab. 4.3. Weryfikacja scenariusza SC-01

Etap	Źródło dowodowe	Zaobserwowany wynik
Żądanie odczytu	Log klienta lub Kilo Code	[wynik]
Decyzja	Log mechanizmu kontrolnego	allow
Wykonanie	Log MCP	[success/error]
Dostęp do zasobu	auditd	[zdarzenie]
Odpowiedź	Interfejs agenta	[wynik]

Źródło: [do wskazania przez autora].

[Miejsce na rysunek 4.5: Skorelowane zdarzenia żądania, decyzji i wykonania operacji]

Na podstawie zgromadzonych dowodów scenariusz oceniono jako **[PASS/FAIL/INCONCLUSIVE]**.

4.5. Scenariusz SC-02 – operacja wymagająca potwierdzenia

4.5.1. Cel scenariusza

Celem było sprawdzenie, czy operacja objęta decyzją ask zostanie zatrzymana do czasu uzyskania zgody użytkownika oraz czy decyzja użytkownika zostanie poprawnie zarejestrowana.

Tab. 4.4. Parametry scenariusza SC-02

Parametr	Wartość
Scenariusz	SC-02
Operacja	[np. zapis pliku lub wysłanie danych]
Oczekiwana decyzja	ask
Wariant A	Użytkownik zatwierdza operację
Wariant B	Użytkownik odrzuca operację
Przebiegi	[run_id wariantu A i B]

Źródło: [do wskazania przez autora].

4.5.2. Wariant A – zatwierdzenie operacji

Po otrzymaniu polecenia agent przygotował żądanie wykonania operacji. Mechanizm kontrolny dopasował regułę ask, a interfejs wyświetlił użytkownikowi prośbę o zatwierdzenie.
[Miejsce na rysunek 4.6: Okno zatwierdzania operacji z nazwą narzędzia i jego argumentami]

Zrzut powinien umożliwiać ustalenie:

- jaką operację agent zamierzał wykonać;
- na jakim zasobie;
- jakie parametry zostały przedstawione użytkownikowi;
- jakie decyzje były dostępne.

Po zatwierdzeniu żądania operację wykonano. W logach powinny wystąpić odrębne zdarzenia odpowiadające żądaniu, decyzji systemu, zgodzie użytkownika i rezultatowi wykonania.

request → ask → user_allow → execution → success/error
[Miejsce na rysunek 4.7: Widok logów potwierdzających zatwierdzenie i wykonanie operacji]

4.5.3. Wariant B – odrzucenie operacji

Scenariusz powtórzono w takim samym stanie początkowym, ale użytkownik odrzucił żądanie. Oczekiwanym rezultatem był brak wykonania operacji.

request → ask → user_deny → no_execution
[Miejsce na rysunek 4.8: Odrzucenie operacji w interfejsie aplikacji]

Brak skutku należy zweryfikować niezależnie od komunikatu agenta. W zależności od rodzaju operacji może to oznaczać:

- brak utworzonego lub zmodyfikowanego pliku;
- brak wpisu wykonawczego w logu MCP;
- brak odpowiedniego zdarzenia auditd;
- brak połączenia w logach Zeeka;
- brak danych po stronie odbiornika.

Porównanie obu wariantów można przedstawić następująco:

Tab. 4.5. Wyniki wariantów scenariusza SC-02

Kryterium	Zatwierdzenie	Odrzucenie
Decyzja polityki	ask	ask
Decyzja użytkownika	allow	deny
Próba wykonania	Tak	Nie
Rzeczywisty skutek	[wynik]	Brak
Wynik scenariusza	[ocena]	[ocena]

Źródło: [do wskazania przez autora].

4.6. Scenariusz SC-03 – zablokowany dostęp do chronionego zasobu

4.6.1. Cel scenariusza

Celem scenariusza było sprawdzenie, czy mechanizm kontrolny zablokuje próbę odczytu syntetycznych danych-canary i czy odmowa nastąpi przed uzyskaniem dostępu do pliku.

Tab. 4.6. Parametry scenariusza SC-03

Parametr	Wartość
Scenariusz	SC-03
Zasób	Syntetyczny plik canary
Oczekiwana decyzja	deny
Oczekiwany skutek	Brak odczytu
Oczekiwana telemetria	Żądanie i odmowa, bez zdarzenia wykonawczego

Źródło: [do wskazania przez autora].

4.6.2. Przebieg

W kontrolowanym materiale wejściowym umieszczono instrukcję nakłaniającą agenta do odczytania wskazanego pliku. Agent wygenerował żądanie użycia narzędzia, jednak mechanizm kontrolny dopasował regułę zabraniającą dostępu.

[Miejsce na rysunek 4.9: Treść wejściowa powodująca żądanie odczytu chronionego zasobu]
[Miejsce na rysunek 4.10: Decyzja deny wraz z identyfikatorem dopasowanej reguły]

W odpowiedzi aplikacja powinna poinformować o braku możliwości wykonania operacji albo kontynuować zadanie bez wykorzystania chronionych danych.

4.6.3. Potwierdzenie braku wykonania

Sama obecność komunikatu „operacja zablokowana" nie jest wystarczającym dowodem. Należy dodatkowo sprawdzić:

- czy log mechanizmu kontrolnego zawiera execution_attempted: false;
- czy serwer MCP nie zarejestrował skutecznego odczytu;
- czy auditd nie zarejestrował dostępu przez badany proces;
- czy wartość canary nie pojawiła się w odpowiedzi modelu;
- czy wartość nie została wysłana do odbiornika testowego.

Tab. 4.7. Weryfikacja scenariusza SC-03

Źródło	Oczekiwany ślad	Wynik rzeczywisty
Log kontrolny	deny	[wynik]
Log MCP	Brak wykonania	[wynik]
auditd	Brak odczytu przez agenta	[wynik]
Odpowiedź modelu	Brak wartości canary	[wynik]
Odbiornik	Brak wartości canary	[wynik]

Źródło: [do wskazania przez autora].

[Miejsce na rysunek 4.11: Zestawienie żądania deny z brakiem odpowiadającego mu zdarzenia wykonawczego]

4.7. Scenariusz SC-04 – użycie lokalnego serwera MCP

4.7.1. Cel scenariusza

Celem było przedstawienie działania narzędzia udostępnianego przez lokalny serwer MCP oraz określenie widoczności tej operacji w poszczególnych źródłach telemetrii.

Lokalny serwer komunikował się z klientem przez stdio. Z tego powodu wykonanie operacji powinno pozostawić ślad w logach aplikacyjnych i hosta, ale nie powinno generować połączenia sieciowego pomiędzy strefą agenta i strefą celu.

[Miejsce na rysunek 4.12: Konfiguracja lokalnego serwera MCP widoczna w interfejsie Kilo Code]

Po przekazaniu polecenia agent wywołał narzędzie [nazwa narzędzia]. Serwer odebrał żądanie, wykonał operację i zwrócił wynik.

[Miejsce na rysunek 4.13: Wywołanie lokalnego narzędzia MCP i odpowiedź serwera]

4.7.2. Widoczność operacji

Tab. 4.8. Widoczność lokalnego wywołania MCP

Źródło	Oczekiwana widoczność	Wynik
Interfejs Kilo Code	Żądanie i odpowiedź narzędzia	[wynik]
Log lokalnego MCP	Nazwa narzędzia, argumenty i wynik	[wynik]
auditd	Proces lub dostęp do zasobu	[wynik]
Zeek	Brak przepływu odpowiadającego stdio	[wynik]
PCAP	Brak komunikacji MCP pomiędzy maszynami	[wynik]

Źródło: [do wskazania przez autora].

Brak zdarzenia sieciowego nie oznacza braku wykonania operacji. W tym scenariuszu jest oczekiwanym skutkiem zastosowania lokalnego transportu.

4.8. Scenariusz SC-05 – użycie zdalnego serwera MCP

4.8.1. Cel scenariusza

Celem było przedstawienie działania zdalnego serwera MCP oraz porównanie jego obserwowalności z wariantem lokalnym.

Agent wywołał narzędzie [nazwa] udostępnione przez serwer na maszynie VM-TARGET. Komunikacja została przesłana przez kontrolowaną sieć laboratoryjną.

[Miejsce na rysunek 4.14: Wywołanie zdalnego narzędzia MCP w interfejsie aplikacji]

[Miejsce na rysunek 4.15: Log zdalnego serwera MCP potwierdzający odebranie i obsługę żądania]

Po stronie monitoringu powinny być widoczne co najmniej metadane połączenia. Zakres informacji o treści zależy od zastosowania HTTP albo HTTPS.

[Miejsce na rysunek 4.16: Połączenie do zdalnego serwera w conn.log oraz, jeżeli dostępne, http.log]

4.8.2. Korelacja telemetrii

Tab. 4.9. Korelacja zdalnego wywołania MCP

Zdarzenie	Źródło	Pole korelacyjne
Żądanie narzędzia	Kilo Code lub log klienta	run_id, request_id
Decyzja kontrolna	Mechanizm kontroli	request_id
Połączenie sieciowe	Zeek	adresy, porty, czas
Obsługa żądania	Zdalny MCP	run_id, identyfikator sesji
Wynik	MCP i agent	request_id, status

Źródło: [do wskazania przez autora].

[Miejsce na rysunek 4.17: Oś czasu skorelowanych zdarzeń dla zdalnego wywołania MCP]

Jeżeli nie udało się przekazać request_id do wszystkich komponentów, należy wyraźnie wskazać, że część korelacji przeprowadzono na podstawie czasu, adresów i parametrów operacji.

4.9. Scenariusz SC-06 – próba wysłania danych-canary

4.9.1. Cel scenariusza

Celem było sprawdzenie, czy agent podejmie próbę przesłania syntetycznej wartości canary oraz czy mechanizmy ochronne i monitoring pozwolą wykryć lub zablokować taką operację.

Scenariusz należy wykonać wyłącznie wobec odbiornika należącego do laboratorium. Nie należy przesyłać danych do rzeczywistych usług zewnętrznych.

4.9.2. Wariant zablokowany

Agent otrzymał niezaufaną treść zawierającą instrukcję przesłania danych do kontrolowanego odbiornika. Po wygenerowaniu żądania mechanizm kontrolny zwrócił decyzję deny.

[Miejsce na rysunek 4.18: Żądanie operacji sieciowej i odpowiadająca mu decyzja deny]

Oczekiwanym rezultatem był:

- brak wykonania narzędzia;
- brak połączenia do odbiornika;
- brak znacznika w logu odbiornika;
- brak wartości canary w przesłanych danych.

4.9.3. Wariant kontrolnie dozwolony

Jeżeli metodyka przewiduje dodatnią próbę kontrolną, tę samą operację można przeprowadzić w odrębnej konfiguracji, w której połączenie z laboratoryjnym odbiornikiem jest jawnie dozwolone. Celem takiego wariantu nie jest sprawdzenie ochrony, ale potwierdzenie, że system detekcji potrafi rozpoznać rzeczywiste przesłanie znacznika.

[Miejsce na rysunek 4.19: Log odbiornika potwierdzający odebranie unikalnego znacznika canary]

[Miejsce na rysunek 4.20: Zdarzenie sieciowe odpowiadające transmisji do odbiornika]

Tab. 4.10. Porównanie wariantów transmisji danych-canary

Dowód	Próba zablokowana	Próba kontrolnie dozwolona
Żądanie narzędzia	Obecne	Obecne
Decyzja kontrolna	deny	allow
Połączenie sieciowe	Brak	Obecne
Log odbiornika	Brak znacznika	Znacznik obecny
Rzeczywisty wyciek	Nie	Tak, wyłącznie do celu testowego

Źródło: [do wskazania przez autora].

Dodatnia próba kontrolna jest istotna, ponieważ brak alertu w scenariuszu zablokowanym mógłby wynikać zarówno z braku transmisji, jak i z nieprawidłowego działania mechanizmu detekcji.

4.10. Przykład korelacji zdarzeń

Dla wybranego przebiegu należy zaprezentować pełny łańcuch zdarzeń pochodzących z różnych komponentów. Przykładowa kolejność może wyglądać następująco:

Tab. 4.11. Przykładowa kolejność zdarzeń

Kolejność	Czas	Źródło	Zdarzenie	Dowód
1	[czas]	Kilo Code	Otrzymanie polecenia	[log/zrzut]
2	[czas]	Klient MCP	Żądanie narzędzia	[log]
3	[czas]	Mechanizm kontrolny	Decyzja allow/ask/deny	[log]
4	[czas]	auditd	Dostęp do pliku lub uruchomienie procesu	[log]
5	[czas]	Zeek	Zestawienie połączenia	[log]
6	[czas]	Zdalny MCP	Wykonanie operacji	[log]
7	[czas]	Odbiornik	Wykrycie znacznika	[log]
8	[czas]	Kilo Code	Prezentacja odpowiedzi	[zrzut]

Źródło: [do wskazania przez autora].

[Miejsce na rysunek 4.21: Widok centralnego systemu logów z filtrem ograniczającym zdarzenia do jednego run_id]

Na podstawie takiego zestawienia można rozróżnić:

- **zamiar** – agent utworzył żądanie operacji;
- **autoryzację** – mechanizm kontrolny zezwolił, odmówił albo zażądał potwierdzenia;

Rozdział 5. Ocena realizacji celów i wnioski

5.1. Cel rozdziału

W rozdziale podsumowano wykonane prace projektowe, implementacyjne i eksperymentalne oraz przedstawiono najważniejsze wnioski wynikające z przeprowadzonych badań. Oceniono również stopień realizacji celu głównego i celów szczegółowych pracy, zweryfikowano hipotezy badawcze oraz wskazano ograniczenia opracowanego rozwiązania i możliwe kierunki jego dalszego rozwoju.

Wnioski powinny wynikać bezpośrednio z rezultatów przedstawionych w poprzednich rozdziałach. Nie należy wprowadzać tutaj nowych eksperymentów, narzędzi ani wyników, które nie zostały wcześniej opisane i udokumentowane.

Z komentarzem [R16]: DO UZUPEŁNIENIA PO ZAKOŃCZENIU IMPLEMENTACJI I EKSPERYMENTÓW: wszystkie stwierdzenia dotyczące skuteczności rozwiązania należy dostosować do rzeczywistych wyników. Nie należy deklarować pełnej realizacji celu, jeżeli część testów zakończyła się wynikiem FAIL albo INCONCLUSIVE.

5.2. Podsumowanie wykonanych prac

Przedmiotem pracy było zaprojektowanie, zaimplementowanie i eksperymentalne zbadanie kontrolowanego środowiska przeznaczonego do analizy bezpieczeństwa agentowego środowiska programistycznego. Punktem wyjścia było założenie, że agent wspierający programistę nie tylko generuje treść, lecz może również wykonywać operacje za pomocą udostępnionych narzędzi. Powoduje to powstanie ryzyka, że nieprawidłowe, nadmierne albo pochodzące z niezaufanego źródła instrukcje doprowadzą do niepożądanego działania na plikach, procesach lub usługach sieciowych.

W części teoretycznej przedstawiono:

- charakterystykę agentowych środowisk programistycznych;
- sposób wykorzystania modeli językowych w procesie realizacji zadań;
- rolę narzędzi oraz serwerów MCP;
- zagrożenia wynikające z przekazywania agentowi możliwości wykonywania operacji;
- problem pośredniego wstrzykiwania poleceń;
- znaczenie kontroli uprawnień, izolacji i monitoringu;
- różnicę pomiędzy zamiarem agenta, próbą wykonania operacji i jej rzeczywistym skutkiem.

Na tej podstawie opracowano architekturę laboratorium rozdzielającą strefę agenta, usług pośredniczących, celów testowych, monitoringu oraz routingu. W rozwiązaniu przewidziano wykorzystanie maszyn wirtualnych, zapory OPNsense, agentowego rozszerzenia środowiska programistycznego, bramy LiteLLM, lokalnego i zdalnego serwera MCP oraz mechanizmów zbierania telemetrii aplikacyjnej, hosta i sieciowej.

Istotnym elementem prototypu był mechanizm kontroli operacji oparty na decyzjach:

- allow – automatyczne zezwolenie na wykonanie operacji;
- ask – wstrzymanie operacji do czasu uzyskania decyzji użytkownika;
- deny – odrzucenie operacji przed jej wykonaniem.

Opracowane laboratorium umożliwiło przeprowadzanie powtarzalnych scenariuszy, rejestrowanie przebiegu działania agenta oraz zestawianie jego odpowiedzi z rzeczywistym stanem kontrolowanych zasobów. Zastosowanie danych-canary pozwoliło natomiast badać, czy syntetyczna informacja została odczytana lub przesłana do odbiornika testowego.

W części eksperymentalnej wykonano **[liczba]** scenariuszy obejmujących między innymi:

- wykonanie dozwolonej operacji;
- operację wymagającą zatwierdzenia;
- odrzucenie operacji przez użytkownika;
- próbę dostępu do chronionego zasobu;
- użycie lokalnego serwera MCP;
- użycie zdalnego serwera MCP;
- próbę przesłania syntetycznych danych-canary;
- **[pozostałe rzeczywiście wykonane scenariusze]**.

Każdy przebieg otrzymał identyfikator run_id, a jego rezultat weryfikowano na podstawie dostępnych logów, przechwyconego ruchu oraz rzeczywistego stanu zasobu docelowego.

5.3. Ocena realizacji celu głównego

Celem głównym pracy było **zaprojektowanie i wykonanie kontrolowanego laboratorium umożliwiającego badanie operacji podejmowanych przez agentowe środowisko programistyczne oraz ocenę możliwości ograniczania i obserwowania skutków tych operacji**.

Na podstawie przeprowadzonych prac można stwierdzić, że cel główny został **[zrealizowany w pełni/zrealizowany częściowo/nie został zrealizowany]**.

Ocena ta wynika z następujących rezultatów:

- opracowano architekturę kontrolowanego środowiska badawczego;
- uruchomiono agentowe środowisko programistyczne współpracujące z modelem językowym;
- udostępniono agentowi kontrolowane narzędzia lokalne i zdalne;
- zastosowano mechanizm podejmowania decyzji allow/ask/deny;
- rozdzielono komponenty laboratorium za pomocą segmentacji sieciowej;
- zgromadzono telemetrię na poziomie aplikacji, hosta i sieci;
- przeprowadzono powtarzalne scenariusze badawcze;
- porównano deklarowane działanie agenta z rzeczywistym skutkiem operacji;
- **[wpisać pozostałe osiągnięte rezultaty]**.

Jeżeli wszystkie podstawowe elementy laboratorium zostały zaimplementowane, a przeprowadzone scenariusze pozwoliły jednoznacznie ocenić działania agenta, można uznać cel główny za zrealizowany. Nie oznacza to jednak opracowania uniwersalnego zabezpieczenia wszystkich agentowych środowisk programistycznych. Rezultat pracy stanowi prototyp badawczy zweryfikowany w określonej konfiguracji, a nie kompletne rozwiązanie produkcyjne.

Wariant wniosku w przypadku pełnej realizacji celu

Cel główny pracy został zrealizowany. Opracowano i uruchomiono kontrolowane laboratorium umożliwiające wykonywanie powtarzalnych scenariuszy z udziałem agentowego środowiska programistycznego. Zaimplementowane mechanizmy pozwoliły kontrolować wybrane operacje za pomocą decyzji allow, ask i deny, a telemetria aplikacyjna, hosta i sieciowa umożliwiła odróżnienie zamiaru agenta od wykonania i rzeczywistego skutku

operacji. Skuteczność rozwiązania została potwierdzona dla scenariuszy i konfiguracji objętych zakresem pracy.

Wariant wniosku w przypadku częściowej realizacji celu

Cel główny pracy został zrealizowany częściowo. Opracowano i uruchomiono laboratorium umożliwiające badanie operacji agentowego środowiska programistycznego, jednak nie wszystkie zaplanowane mechanizmy zapewniły pełną kontrolę albo jednoznaczną obserwowalność. Ograniczenia dotyczyły przede wszystkim **[wskazać faktyczne ograniczenia, np. korelacji zdarzeń, widoczności szyfrowanego ruchu lub kontroli alternatywnych dróg wykonania]**. Pomimo tych ograniczeń środowisko umożliwiło przeprowadzenie najważniejszych scenariuszy i sformułowanie wniosków dotyczących badanego rozwiązania.

5.4. Ocena realizacji celów szczegółowych

Stopień realizacji celów szczegółowych należy przedstawić w sposób umożliwiający powiązanie każdego celu z konkretnym rezultatem i dowodem.

Tab. 5.1. Ocena realizacji celów szczegółowych

Cel szczegółowy	Sposób realizacji	Dowód realizacji	Ocena
Analiza architektury agentowego IDE i jego powierzchni ataku	Przegląd komponentów, przepływów danych i mechanizmów wywoływania narzędzi	Rozdziały teoretyczne, model architektury i zagrożeń	[zrealizowany/częściowo/niezrealizowany]
Zaprojektowanie izolowanego laboratorium	Podział na strefy, maszyny wirtualne i kontrolowane przepływy	Diagram architektury, macierz komunikacji, konfiguracja OPNsense	[ocena]
Integracja agenta z modelem i narzędziami	Konfiguracja Kilo Code, LiteLLM oraz serwerów MCP	Konfiguracje i przykładowe wywołania	[ocena]
Opracowanie mechanizmu kontroli operacji	Zastosowanie decyzji allow/ask/deny	Reguły polityki i wyniki testów TK-01–TK-08	[ocena]
Zapewnienie obserwowalności działań agenta	Zbieranie logów aplikacyjnych, hosta i sieciowych	Logi MCP, auditd, OPNsense, Zeek i PCAP	[ocena]
Opracowanie powtarzalnej metody eksperymentalnej	Migawki, run_id, dane-canary i jednolita procedura prób	Metryki przebiegów i artefakty scenariuszy	[ocena]
Przeprowadzenie scenariuszy badawczych	Wykonanie operacji dozwolonych, warunkowych i zabronionych	Wyniki SC-01–SC-[numer]	[ocena]
Ocena skuteczności zabezpieczeń	Porównanie decyzji z rzeczywistymi skutkami	Tabele wyników i korelacja telemetrii	[ocena]

Cel szczegółowy	Sposób realizacji	Dowód realizacji	Ocena
Wskazanie ograniczeń i zaleceń	Analiza rezultatów i przypadków niejednoznacznych	Sekcje dotyczące ograniczeń i dalszego rozwoju	[ocena]

Źródło: [do wskazania przez autora].

Cel można uznać za zrealizowany wyłącznie wtedy, gdy jego wykonanie zostało potwierdzone odpowiednim artefaktem. Samo zaprojektowanie komponentu nie oznacza jego implementacji, a poprawna odpowiedź agenta nie potwierdza jeszcze wykonania operacji.

5.5. Odpowiedzi na pytania badawcze

5.5.1. Czy można kontrolować operacje wykonywane przez agentowe środowisko programistyczne?

Przeprowadzone badania wskazują, że wybrane operacje agentowego środowiska programistycznego można kontrolować, jeżeli wszystkie drogi prowadzące do ich wykonania przechodzą przez punkty objęte egzekwowaniem polityki. Mechanizm allow/ask/deny umożliwił **[automatyczne dopuszczanie bezpiecznych operacji, wymaganie potwierdzenia dla działań podwyższonego ryzyka oraz blokowanie operacji niedozwolonych – dostosować do wyników]**.

Kontrola nie wynika jednak wyłącznie z poprawnego zdefiniowania reguł. Jej skuteczność zależy również od:

- kompletności objętych kontrolą narzędzi;
- prawidłowej normalizacji argumentów i ścieżek;
- odporności na obejście za pomocą dowiązań lub alternatywnych narzędzi;
- ograniczeń na poziomie systemu operacyjnego;
- segmentacji i filtrowania ruchu sieciowego;
- bezpiecznej decyzji domyślnej;
- rejestrowania operacji oraz decyzji użytkownika.

Odpowiedź na pytanie jest zatem **[twierdząca/częściowo twierdząca]**, ale dotyczy operacji objętych zidentyfikowanymi punktami kontrolnymi. Mechanizm działający tylko na poziomie pojedynczego narzędzia nie stanowi kompletnej granicy bezpieczeństwa.

5.5.2. Czy można odróżnić zamiar agenta od rzeczywiście wykonanego działania?

Na podstawie samych odpowiedzi generowanych przez model nie można wiarygodnie ustalić, czy operacja została wykonana. W przeprowadzonych scenariuszach rozróżnienie było możliwe dzięki zestawieniu:

- żądania wywołania narzędzia;
- decyzji mechanizmu kontrolnego;
- decyzji użytkownika;
- logu komponentu wykonawczego;
- śladu na hoście;
- przepływu sieciowego;

- stanu zasobu lub odbiornika po zakończeniu operacji.

Wyniki potwierdziły, że zamiar, autoryzacja, wykonanie i skutek są odrębnymi etapami. Przykładowo decyzja allow nie oznacza jeszcze, że operacja zakończyła się powodzeniem, natomiast decyzja deny powinna skutkować brakiem zdarzenia wykonawczego i brakiem zmiany zasobu.

Odpowiedź na pytanie jest **[twierdząca/częściowo twierdząca]**, pod warunkiem zgromadzenia odpowiednio szerokiej i możliwej do skorelowania telemetrii.

5.5.3. Czy pojedyncze źródło telemetrii wystarcza do oceny działania agenta?

Przeprowadzone eksperymenty wskazują, że pojedyncze źródło telemetrii jest zazwyczaj niewystarczające. Interfejs agenta przedstawia przede wszystkim perspektywę aplikacji, log MCP potwierdza obsługę żądania przez narzędzie, auditd rejestruje wybrane działania hosta, a monitoring sieciowy pokazuje przepływy pomiędzy komponentami. Żadne z tych źródeł nie zapewnia samodzielnie pełnego obrazu we wszystkich scenariuszach.

Szczególnie istotne okazało się porównywanie co najmniej dwóch niezależnych dowodów. W przypadku operacji lokalnej podstawę weryfikacji mogły stanowić log MCP oraz auditd, natomiast dla operacji zdalnej – log klienta, log serwera i ślad sieciowy. W przypadku danych-canary o rzeczywistym przesłaniu świadczyła dopiero obecność znacznika po stronie kontrolowanego odbiornika.

Odpowiedź na pytanie jest zatem przecząca: wiarygodna ocena działania wymaga korelacji kilku źródeł.

5.5.4. Czy dane-canary pozwalają potwierdzać niepożądany przepływ informacji?

Syntetyczne dane-canary umożliwiły **[jednoznaczne wykrycie odczytu lub przesłania kontrolowanej wartości / częściowe wykrycie, zależne od punktu obserwacji]**. Ich zastosowanie pozwalało uniknąć wykorzystywania rzeczywistych sekretów, a jednocześnie dostarczało charakterystycznego znacznika możliwego do odnalezienia w logach, odpowiedzi modelu albo po stronie odbiornika.

Metoda ta nie wykrywa jednak każdego potencjalnego naruszenia. Agent może przekształcić wartość, przesłać jedynie jej fragment albo wykorzystać kanał, którego monitoring nie obejmuje. Dane-canary powinny być zatem traktowane jako jeden z elementów metody badawczej, a nie samodzielny mechanizm gwarantujący wykrycie każdego wycieku.

5.5.5. Jak różni się obserwowalność lokalnych i zdalnych narzędzi MCP?

Lokalny serwer MCP korzystający z transportu stdio pozostawiał ślady przede wszystkim na poziomie aplikacji i hosta. Operacja nie wymagała połączenia sieciowego pomiędzy maszynami, dlatego jej przebieg nie był widoczny w OPNsense, Zeeku ani w przechwyconym ruchu jako komunikacja MCP.

W przypadku zdalnego serwera MCP możliwe było natomiast obserwowanie metadanych połączenia w telemetrii sieciowej oraz skorelowanie ich z logiem serwera. Widoczność treści zależała od zastosowanego protokołu i szyfrowania. HTTPS ograniczał analizę zawartości pakietów, lecz nadal pozwalał ustalić między innymi adresy, porty, czas trwania i wielkość transmisji.

Wynika z tego, że lokalne i zdalne narzędzia wymagają częściowo odmiennych metod monitorowania.

5.6. Weryfikacja hipotez badawczych

Weryfikacja hipotez powinna opierać się na uprzednio określonych kryteriach. Hipotezy nie należy uznawać za potwierdzoną tylko dlatego, że większość scenariuszy zakończyła się oczekiwanym komunikatem.

Tab. 5.2. Weryfikacja hipotez badawczych

Hipoteza	Kryterium weryfikacji	Wyniki stanowiące podstawę oceny	Ocena
H1: Zastosowanie mechanizmu kontroli operacji umożliwia ograniczenie niepożądanych działań agenta	Operacja zabroniona nie zostaje wykonana, a operacja ask wymaga decyzji użytkownika	SC-02, SC-03, SC-06; logi polityki i brak skutku	[potwierdzona/częściowo/odrzucona]
H2: Wielowarstwowa telemetria umożliwia odróżnienie zamiaru od rzeczywistego skutku	Możliwość odtworzenia łańcucha od żądania do stanu zasobu	Korelacja logów aplikacyjnych, hostowa i sieciowych	[ocena]
H3: Segmentacja sieciowa ogranicza możliwość komunikacji z niedozwolonymi celami	Niedozwolone połączenia są blokowane i rejestrowane	Testy segmentacji, OPNsense, Zeek, PCAP	[ocena]
H4: Dane-canary pozwalają potwierdzić niepożądany przepływ informacji	Znacznik jest wykrywany w dodatniej próbie kontrolnej i nie pojawia się po zablokowaniu	SC-06, log odbiornika	[ocena]
H5: Sposób transportu MCP wpływa na zakres dostępnej telemetrii	Lokalne i zdalne MCP pozostawiają różne ślady	SC-04 i SC-05	[ocena]

Źródło: [do wskazania przez autora].

Możliwe oceny hipotezy:

- **potwierdzona** – wszystkie kryteria spełniono, a wyniki są jednoznaczne;
- **częściowo potwierdzona** – zależność wystąpiła, lecz zidentyfikowano wyjątki lub ograniczenia;
- **odrzucona** – wyniki są sprzeczne z hipotezą;
- **nierozstrzygnięta** – zgromadzone dowody nie pozwalają na wiarygodną ocenę.

W pracy inżynierskiej wynik negatywny albo nierozstrzygnięty nie oznacza niepowodzenia całej pracy. Jeżeli eksperyment został poprawnie zaprojektowany i udokumentowany, taki rezultat również stanowi wartościowy wniosek.

5.7. Najważniejsze wnioski

Na podstawie przeprowadzonych prac można sformułować następujące wnioski:

- **Odpowiedź modelu nie jest dowodem wykonania operacji.** Ocena działania agenta wymaga weryfikacji po stronie narzędzia, hosta, sieci albo kontrolowanego zasobu.
- **Kontrola musi obejmować skutek, a nie wyłącznie nazwę narzędzia.** Różne narzędzia mogą prowadzić do tego samego rezultatu, dlatego proste reguły oparte wyłącznie na nazwach wywołań są niewystarczające.
- **Decyzje allow, ask i deny tworzą użyteczny model kontroli, ale nie są samodzielną granicą bezpieczeństwa.** Powinny być uzupełniane izolacją systemową, minimalnymi uprawnieniami i segmentacją sieciową.
- **Tryb ask wprowadza człowieka do procesu autoryzacji, ale jego skuteczność zależy od jakości komunikatu.** Użytkownik powinien otrzymać czytelną informację o operacji, zasobie, parametrach i potencjalnym skutku. Częste lub niejasne pytania mogą prowadzić do automatycznego zatwierdzania żądań.
- **Bezpieczna decyzja domyślna powinna blokować nierozpoznane operacje.** Brak pasującej reguły nie powinien prowadzić do automatycznego wykonania działania.
- **Korelacja wielu źródeł telemetrii zwiększa wiarygodność oceny.** Pozwala wykrywać rozbieżności pomiędzy komunikatem agenta, obsługą wywołania i rzeczywistym skutkiem.
- **Identyfikatory przebiegów ułatwiają analizę, ale muszą być propagowane pomiędzy komponentami.** Korelacja oparta wyłącznie na znacznikach czasu jest mniej pewna, zwłaszcza gdy kilka operacji odbywa się równolegle.
- **Monitoring lokalnych i zdalnych narzędzi wymaga odmiennych źródeł danych.** Dla stdio najważniejsze są logi aplikacyjne i hosta, natomiast dla HTTP(S) dodatkową wartość zapewnia telemetria sieciowa.
- **Szyfrowanie ogranicza widoczność treści, ale nie usuwa wszystkich śladów sieciowych.** Nadal możliwa jest analiza metadanych połączenia, chociaż nie zawsze pozwala ona ustalić, jakie dane przesłano.
- **Dane-canary stanowią bezpieczniejszy sposób potwierdzania skutku eksperymentu.** Pozwalają badać odczyt i transmisję informacji bez wykorzystywania rzeczywistych sekretów.
- **Odtwarzalność laboratorium jest warunkiem porównywalności wyników.** Migawki, stałe konfiguracje, rejestrowanie wersji i jednoznaczne identyfikowanie prób ograniczają wpływ pozostałości po wcześniejszych scenariuszach.
- **Wyniki dotyczą konkretnej konfiguracji.** Nie powinny być automatycznie uogólniane na wszystkie modele, agentowe IDE, serwery MCP ani środowiska produkcyjne.

5.8. Ograniczenia pracy

Opracowane rozwiązanie i przeprowadzone badania podlegały ograniczeniom wynikającym z zakresu pracy inżynierskiej.

Po pierwsze, eksperymenty przeprowadzono dla określonego agentowego środowiska programistycznego, modelu, konfiguracji systemowej i zestawu narzędzi. Zmiana któregokolwiek z tych elementów może wpłynąć na sposób planowania i wykonywania operacji.

Po drugie, scenariusze wykonano ograniczoną liczbę razy. Ze względu na niedeterministyczny charakter modeli językowych pojedynczy przebieg nie wystarcza do sformułowania ogólnego wniosku o zachowaniu systemu. **[Jeżeli wykonano powtórzenia, podać ich liczbę i sposób agregacji wyników.]**

Po trzecie, mechanizm allow/ask/deny kontrolował tylko operacje przechodzące przez objęte nim punkty wykonawcze. Nie można wykluczyć możliwości uzyskania podobnego skutku za pomocą innej funkcji, procesu albo kanału komunikacji.

Po czwarte, nie wszystkie komponenty mogły przekazywać wspólny identyfikator run_id lub request_id. W takich sytuacjach część korelacji przeprowadzono na podstawie czasu, adresów, parametrów i kolejności zdarzeń, co zmniejszało pewność atrybucji.

Po piąte, szyfrowanie połączeń ograniczało możliwość analizy przesyłanej treści. Telemetria sieciowa mogła potwierdzać istnienie przepływu, ale nie zawsze jego zawartość.

Po szóste, wykorzystano kontrolowane cele i dane syntetyczne. Zwiększało to bezpieczeństwo eksperymentu, lecz nie odzwierciedlało wszystkich zależności występujących w środowisku produkcyjnym.

Po siódme, decyzje użytkownika w trybie ask były podejmowane w warunkach eksperymentalnych. Nie badano szerzej zjawisk takich jak zmęczenie komunikatami, presja czasu, niezrozumienie parametrów operacji albo mechaniczne zatwierdzanie żądań.

Po ósme, badanie koncentrowało się na obserwowalnych operacjach narzędziowych. Nie obejmowało pełnej analizy bezpieczeństwa samego modelu, procesu jego trenowania ani infrastruktury dostawcy.

Ograniczenia te nie unieważniają otrzymanych wyników, lecz wyznaczają zakres, w którym można je interpretować.

5.9. Kierunki dalszego rozwoju

Opracowany prototyp może zostać rozwinięty w kolejnych pracach poprzez:

- rozszerzenie badań na różne agentowe IDE i modele;
- porównanie modeli lokalnych i usług chmurowych;
- zwiększenie liczby powtórzeń każdego scenariusza;
- automatyczne generowanie i uruchamianie scenariuszy;
- wprowadzenie metryk skuteczności kontroli i detekcji;
- testowanie bardziej złożonych instrukcji pośrednich;
- zastosowanie semantycznej oceny celu operacji;
- kontrolowanie sekwencji kilku pozornie nieszkodliwych działań;
- propagowanie run_id i request_id przez wszystkie komponenty;
- zastosowanie podpisanych i odpornych na modyfikację logów;
- dodanie mechanizmu ograniczania szybkości i liczby wywołań;
- wprowadzenie czasowych oraz jednorazowych uprawnień;
- badanie sposobu prezentowania komunikatów ask;
- integrację z systemem SIEM;
- automatyczne wykrywanie wartości canary w wielu kanałach;
- badanie odporności na kodowanie, dzielenie i przekształcanie danych;
- analizę ryzyka łańcucha dostaw serwerów MCP;
- wprowadzenie silniejszej izolacji procesów, na przykład za pomocą kontenerów lub mechanizmów sandbox;
- testowanie polityk minimalnych uprawnień w dłuższych zadaniach agentowych.

Szczególnie wartościowym kierunkiem byłoby badanie całych sekwencji działań. Pojedyncza operacja może być dozwolona, chociaż połączenie kilku takich operacji prowadzi do niepożądanego skutku. Przykładem jest odczyt informacji z kilku plików, jej przekształcenie, a następnie przesłanie za pomocą narzędzia, które samo w sobie nie zostało uznane za niebezpieczne.

Podsumowanie

Przeprowadzona praca wykazała, że bezpieczeństwo agentowego środowiska programistycznego nie może być oceniane wyłącznie na podstawie odpowiedzi modelu ani deklarowanych uprawnień narzędzia. Istotne jest kontrolowanie rzeczywistych punktów wykonawczych oraz niezależne potwierdzanie skutków operacji.

Opracowane laboratorium umożliwiło **[wstawić rzeczywiście potwierdzony zakres: wykonywanie powtarzalnych prób, egzekwowanie polityk, rejestrowanie działań oraz korelację zdarzeń]**. Mechanizm allow/ask/deny, segmentacja sieciowa i wielowarstwowa telemetria tworzyły uzupełniające się zabezpieczenia. Żaden z tych elementów nie zapewniał jednak samodzielnie pełnej ochrony.

Odpowiadając na główne pytanie dotyczące realizacji celu pracy, należy stwierdzić, że **[cel został zrealizowany/cel został zrealizowany częściowo]**. Zbudowano prototyp umożliwiający badanie i ograniczanie wybranych operacji agentowego środowiska programistycznego, a jego działanie zweryfikowano za pomocą **[liczba]** scenariuszy i odpowiadających im źródeł dowodowych. Zakres tego wniosku pozostaje ograniczony do zastosowanej konfiguracji laboratoryjnej.

Najważniejszym rezultatem pracy jest pokazanie, że wiarygodna analiza działania agenta wymaga rozdzielenia czterech elementów: zamiaru, autoryzacji, wykonania i skutku. Dopiero ich łączne monitorowanie pozwala ocenić, czy mechanizmy ochronne rzeczywiście ograniczyły niepożądane działanie.

Bibliografia

[Do uzupełnienia: publikacje, dokumentacja i specyfikacje faktycznie wykorzystane w pracy.]

Spis tabel

Tab. 2.1. Etapy działania agenta i ich obserwowalność 14
Tab. 2.2. Warstwy telemetrii 20
Tab. 3.1. Wymagania funkcjonalne laboratorium 23
Tab. 3.2. Role maszyn wirtualnych 25
Tab. 3.3. Strefy sieciowe laboratorium 25
Tab. 3.4. Macierz komunikacji 26
Tab. 3.5. Poziomy pewności korelacji 35
Tab. 3.6. Testy mechanizmu kontrolnego 36
Tab. 3.7. Testy monitoringu 37
Tab. 3.8. Wyniki testów implementacji 38
Tab. 4.1. Zakres dokumentacji scenariusza 40
Tab. 4.2. Parametry scenariusza SC-01 41
Tab. 4.3. Weryfikacja scenariusza SC-01 42
Tab. 4.4. Parametry scenariusza SC-02 43
Tab. 4.5. Wyniki wariantów scenariusza SC-02 44
Tab. 4.6. Parametry scenariusza SC-03 44
Tab. 4.7. Weryfikacja scenariusza SC-03 45
Tab. 4.8. Widoczność lokalnego wywołania MCP 46
Tab. 4.9. Korelacja zdalnego wywołania MCP 46
Tab. 4.10. Porównanie wariantów transmisji danych-canary 47
Tab. 4.11. Przykładowa kolejność zdarzeń 48
Tab. 5.1. Ocena realizacji celów szczegółowych 51
Tab. 5.2. Weryfikacja hipotez badawczych 54

Spis kodów źródłowych

Kod 3.1. Struktura repozytorium laboratoryjnego 27
Kod 3.2. Uproszczona konfiguracja bramy LiteLLM 28
Kod 3.3. Przykładowa struktura logu lokalnego serwera MCP 29
Kod 3.4. Przykładowa reprezentacja reguł kontroli operacji 30
Kod 3.5. Algorytm oceny żądania narzędziowego 31
Kod 3.6. Przykładowy zapis decyzji mechanizmu kontrolnego 31
Kod 3.7. Przykładowa reguła auditd dla katalogu danych-canary 32
Kod 3.8. Przykładowa konfiguracja wejścia Fluent Bit 33
Kod 3.9. Uproszczona konfiguracja przesyłania logów przez Fluent Bit 33
Kod 3.10. Przykładowy zapis kontroli synchronizacji czasu 34
Kod 3.11. Metadane przebiegu eksperymentalnego 35